软件工程与方法

第1讲 软件与软件工程

本章内容

- 软件工程中常见问题
- 软件工程的基本概念
- 软件生存期模型
- 软件工程知识体系

第1讲　软件与软件工程

1.1 软件工程中常见问题

软件工程的基本问题

Question	Answer
What is software?	
What are the attributes of good software?	
What is software engineering?	
What are the fundamental software engineering activities?	
What is the difference between software engineering and computer science?	
What is the difference between software engineering and system engineering?	

（续）

Question	Answer
What are the main uncertainties in software development?	
What are the costs of software engineering?	
What is software security? How to develop a secure software?	
What are the best software engineering techniques and methods?	
What are the key challenges facing software engineering?	

软件（Software）

【传统的IEEE标准】软件是一组形成“配置”的项目或对象，其中包括

- 程序
- 文档
- 数据...

Software is a set of items or objects that form a “configuration” that includes

- programs
- documents
- data ...



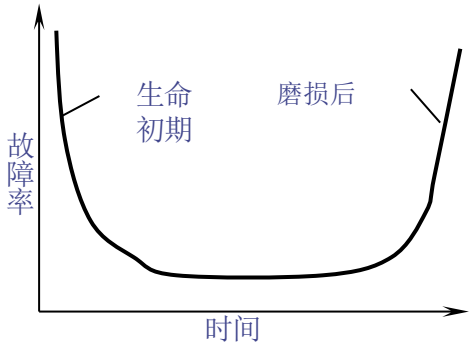
【现代或*未来*的定义】软件 = 程序 + 文档 + 数据 + **AI智能资产**

AI 智能资产包含：

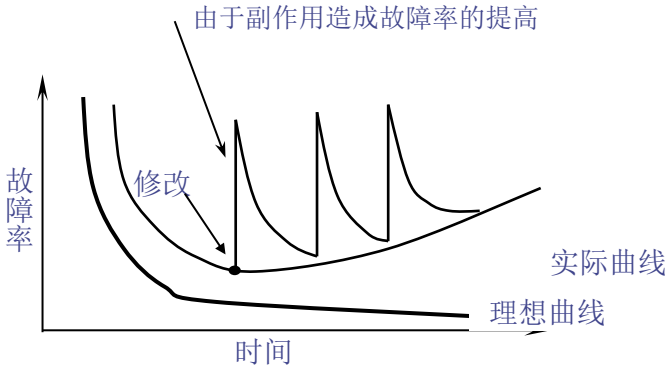
- 各类大模型 / 小模型、多模态模型权重文件；
- Agent 智能体、任务编排引擎；
- RAG 检索知识库、向量库、记忆管理模块；
- 模型护栏、评估规则、微调流水线（MLOps 配套产物）

软件的特点：

1) 逻辑实体，具有抽象性
2) 软件开发过程中没有明显的制造过程
3) 不会“磨损”，但会“退化”
4) 依赖性，依赖硬件与运行环境
5) 软件的复杂性
6) 全生命周期中维护成本远高于开发成本
7) 社会性
8) 软件开发至今尚未完全摆脱手工的开发方式。
9) AI时代的新特点：概率性与动态演进



硬件的故障率曲线



软件的故障率曲线(实际情况下)

2024年微软全球蓝屏事件

CrowdStrike公司的软件更新与微软系统不兼容，导致全球850万台设备蓝屏，直接经济损失超10亿美元。具体影响：

- 航空业：达美航空取消5000+航班，损失5亿美元（含赔偿、酒店费用）
- 企业瘫痪：特斯拉生产线停工，荷兰、澳大利亚机场自助登机系统瘫痪，英国超市关闭。
- 社会成本：医疗病例调取延迟、媒体停播、金融市场短暂混乱。

软件的分类

- 按功能
- 按规模
- 按授权和分发方式
- 按运行架构
- 按应用场景
- 《软件产品分类》（GB/T 36475—2018）



中国软件产品主要分为以下六大类：

① 系统软件

② 支撑软件

③ 应用软件

④ 嵌入式软件

⑤ 信息安全软件

⑥ 工业软件

软件复杂度

- 定义：

  软件复杂度（Software complexity）是软件系统/模块在设计、开发、维护中难以理解、分析与修改的程度，是软件质量与维护成本的核心指标。

- 从度量方式和产生原因两个维度分类：
  - **本质复杂度**（Essential Complexity）（问题域固有，不可消除）
  - **偶然复杂度**（Accidental Complexity）（实现 / 架构引入，可优化）。
- 两个要点：
  - **难以理解 (难以维护和扩展）**
  - **无法预测行为**

研究软件复杂度的角度

- 从度量方式：
  - **静态复杂度**：不运行程序即可分析的复杂度，如圈复杂度（Cyclomatic Complexity）、认知复杂度（Cognitive Complexity）、代码行数（LOC）等。
  - **动态复杂度**：通过程序运行时行为分析的复杂度，如代码覆盖率、异常处理频率、动态依赖关系等。
- 从产生原因：
  - **结构复杂性**：源于代码结构设计（如模块划分、函数嵌套、循环/条件语句），是最常见的复杂度类型。
  - **功能复杂性**：源于代码实现的功能本身（如复杂算法、多业务逻辑），即使结构清晰，功能复杂也会导致理解难度上升

复杂度主要体现在以下方面：

① 认知复杂性（理解难度大）
② 依赖复杂性（耦合度高）
③ 结构与逻辑复杂性（控制流复杂）
④ 规模复杂性（体量庞大）
⑤ 数据复杂性（数据结构纠缠）
⑥ 系统与需求复杂性（外部环境与业务）

常见的度量指标：

- 圈复杂度 (Cyclomatic Complexity, CC)
- 认知复杂度 (Cognitive Complexity)
- Halstead 度量 (Halstead Metrics)
- 依赖关系

软件复杂度为什么很重要

- 缺陷率：
  - 研究表明，复杂度高的模块往往包含更多的Bug。
- 维护成本：
  - 高复杂度使得新加入的开发者难以理解代码，导致“修好一个Bug引出两个新Bug”。
- 技术债务（Technical debt ）：
  - 忽视复杂度会导致技术债务累积，最终可能导致系统需要推倒重来。
- 测试难度：
  - 高圈复杂度意味着需要更多的测试用例才能覆盖所有路径。

降低和管理复杂度：

- 遵循设计原则
- 持续重构 (Refactoring)
- 模块化与抽象
- 自动化监控
- 使用设计模式/领域驱动设计（DDD）



第1讲　软件与软件工程

1.2 什么是软件工程

什么是软件工程

- 软件工程是一门工程学科，涉及软件生产的各个方面，从最初的系统描述一直到使用后的系统维护，都属于其学科范畴。
- 在软件工程的定义中有两个关键词：
  - 工程学科（Engineering discipline）
  - 软件生产的各个方面（all aspects of software production）

软件工程诞生的导火索：

IBM OS/360系统，共有4000多个模块，约100万条指令，投入5000人年，耗资数亿美元，结果还是延期交付。在交付后的系统一中仍发现大量（2000个以上）的错误。每次发行的新版本都是从前一版本中找出1000个程序错误。

布鲁克斯（Frederick P. Brooks）在《人月神话》一书中所描述的场景：

软件开发被喻为让众多史前巨兽痛苦挣扎，却无力摆脱的焦油坑。

软件工程的各种定义：

- Fritz Bauer在1969年NATO会议上的定义
  - 为了经济地获得软件，这个软件是可靠的并且能在实在的计算机上工作，所需要的健全的工程原理（方法）的确立和使用。
- P. Wegner & B. Boehm认为：
  - 运用现代科学知识在设计和构造计算机程序，以及开发、运行和维护这些程序所要求的有关文档编制中的实际应用。
- IEEE在软件汇编术语中定义：
  - 对软件开发、运作、维护的系统化的、有规范的、可定量的方法之应用，即是对软件的工程化应用。

软件工程定义：

软件工程是指导计算机软件开发和维护的工程学科。采用工程的概念、原理、技术和方法来开发与维护软件，把经过时间考验而证明正确的管理技术和当前能够得到的最好的技术方法结合起来，以经济地开发出高质量的软件并有效地维护它，这就是软件工程。

软件工程的特点：

- 工程是将理论和知识应用于**实践的科学**。
- 软件工程借鉴了传统工程的原则和方法，以求**高效地开发高质量软件**。
- 软件工程中应用了计算机科学、数学和管理科学
  - **计算机科学**和数学用于构造模型与算法
  - **工程科学**用于制定规范、设计范型、评估成本及确定权衡
  - **管理科学**用于计划、资源、质量和成本的管理。

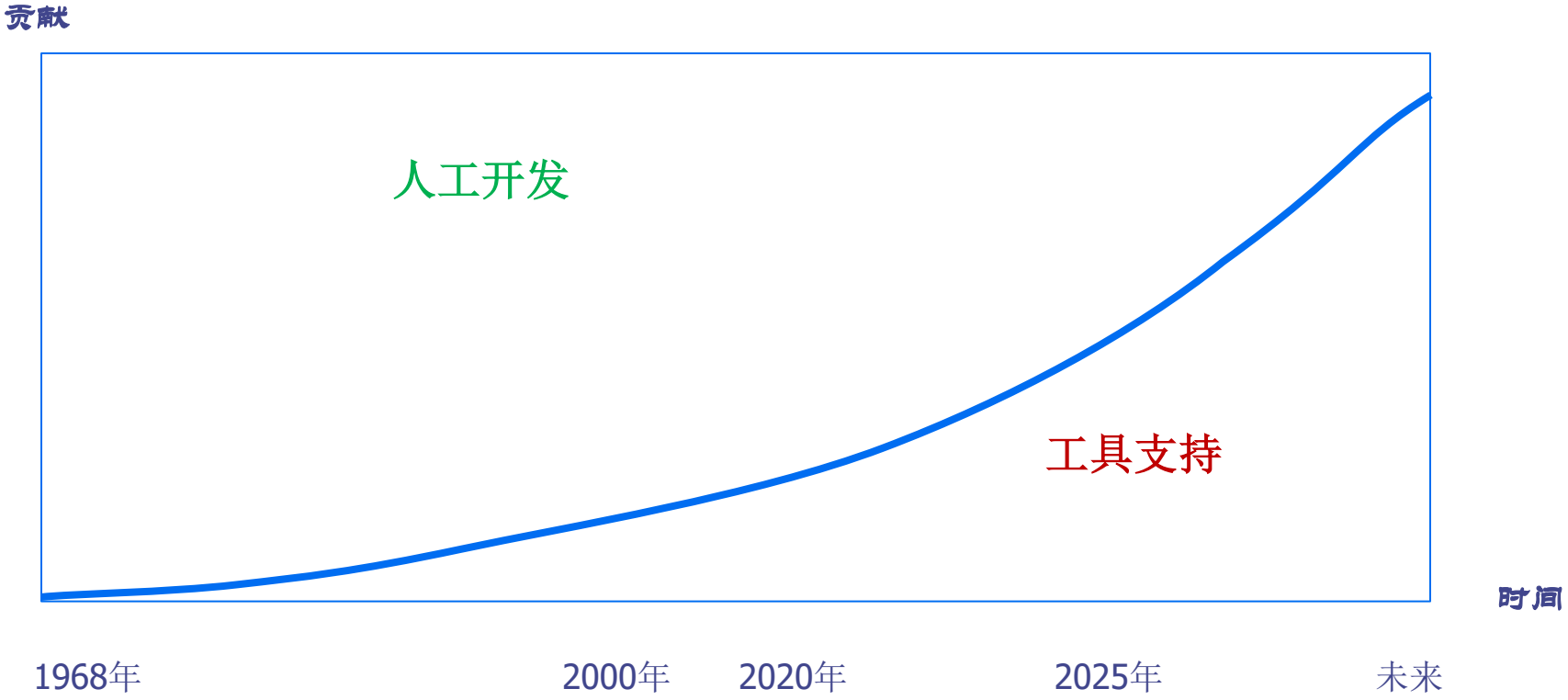
软件工程与一般工程的差别

- 软件是**逻辑产品**，不会损坏、磨损、老化，而且可以不断地改进、优化，其可靠性由逻辑性所决定。
- 由于软件的逻辑产品特性，它的功能只能依赖于**硬件**和**运行环境**以及人们对它的**操作**，才能得以体现。
- 对软件产品的要求比一般有形产品复杂：
  - 用户难以清晰、准确地表达软件产品的功能。
  - 对软件产品的要求，如可靠性、易移植性、易使用性等等是隐含的，也是难以表达的，而且也缺少度量的具体标准，与有形产品的质量检验的精度相距甚远。
  - 软件设计不仅仅涉及到技术复杂性，也不涉及到管理复杂性。
- 在软件设计中的复杂性，引起的软件特征包括四方面：
  - 功能多样性（functional diversity）
  - 实现多样性（implementation diversity）
  - 能见度低（low visibility）
  - 软件结构的合理性差（poor rationality of software structure）



- 任何一种工程，在其年轻时代总是人工密集的，而到其成熟时期则成为资金密集的。软件工程是智力密集。

软件开发方法的演进



软件工程的历史：抽象层次的不断跃迁

- **第一个黄金时代（约20世纪40年代末至70 年代末）的核心是算法抽象。**
  - 面对的主要问题是，如何将现实世界的业务流程或数学问题翻译成机器可以理解的指令序列。
  - 当算法抽象的能力达到极限时，“软件危机”爆发了。
- **第二个黄金时代（约20世纪80年代至21世纪初）的核心是面向对象抽象。**
  - 为了解决「软件危机」，出现新的开发工具。
- **第三个黄金时代：核心抽象是系统、平台与 AI 智能体**
  - 开发者不再从零开始构建一切。他们在一个由云服务商（如 AWS）、大型开源项目（如 Kubernetes）、以及无数 API 构成的巨大平台上进行创作。

软件开发的核心

不是简单的“写代码”，而是一系列认知转换与工程化活动的集合。



软件工程的核心，是在各种约束之间取得平衡的决策艺术。这些约束包括技术上的（物理定律、算法复杂度）、经济上的（预算、时间）、人力上的（团队规模、技能）、以及越来越重要的伦理和社会约束。一个软件工程师的工作，是在这个复杂的多维空间中找到一个「足够好」的解。

软件开发的终极目标是构建一个能够随着业务和环境持续、安全、高效演进的“数字生命体”。

软件开发的本质

软件开发就是建立问题域到开发平台之间的映射。

- 软件开发的目标是将问题域中的概念映射到运行平台的概念，把问题域中的处理逻辑映射到运行平台的处理逻辑。
- 软件开发就是要“弥补”问题域与运行平台之间的“距离”。

软件开发的本质是问题域向运行平台的映射存在一定的复杂性，为了控制这个复发性，需要确定多个抽象层，例如需求、设计、实现和部署等。每个抽象层均有自己的特定的术语定义，并形成该抽象层的一个术语空间。

软件开发研究的问题

- 软件开发涉及两个方面的问题：
  - 如何**实现多个抽象之间的映射**
    - 过程方向：过程活动的组织方式
    - 过程途径：求解软件的开发手段
  - 如何**管理这些抽象之间的映射**
    - 软件项目规划、组织、人员安排、控制和领导等
- 软件开发既有技术问题，又有管理问题。

软件工程主要研究的内容

① 做哪些映射，既要进行哪些开发任务；

② 如何根据软件项目特点、环境因素等，选择并组织这些开发任务；

③ 如何实现不同抽象层之间的映射；

④ 如何支持验证，如何支持整个软件的开发；

⑤ 如何管理一个软件项目；

⑥ 如何支持软件的维护和进化。

软件生产规模增大而复杂度增大

以NASA的软件系统为例：

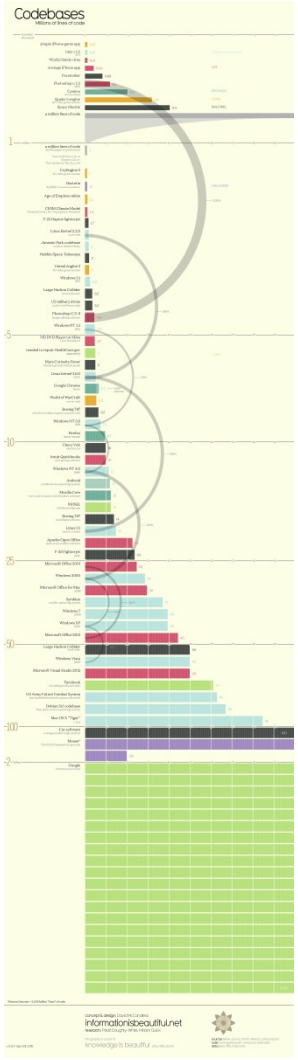
- 1963年水星计划系统 200万条指令
- 1967年双子星座计划系统 400万条指令
- 1973年阿波罗计划系统 1000万条指令
- 1979年哥伦比亚航天飞机系统 4000万条指令

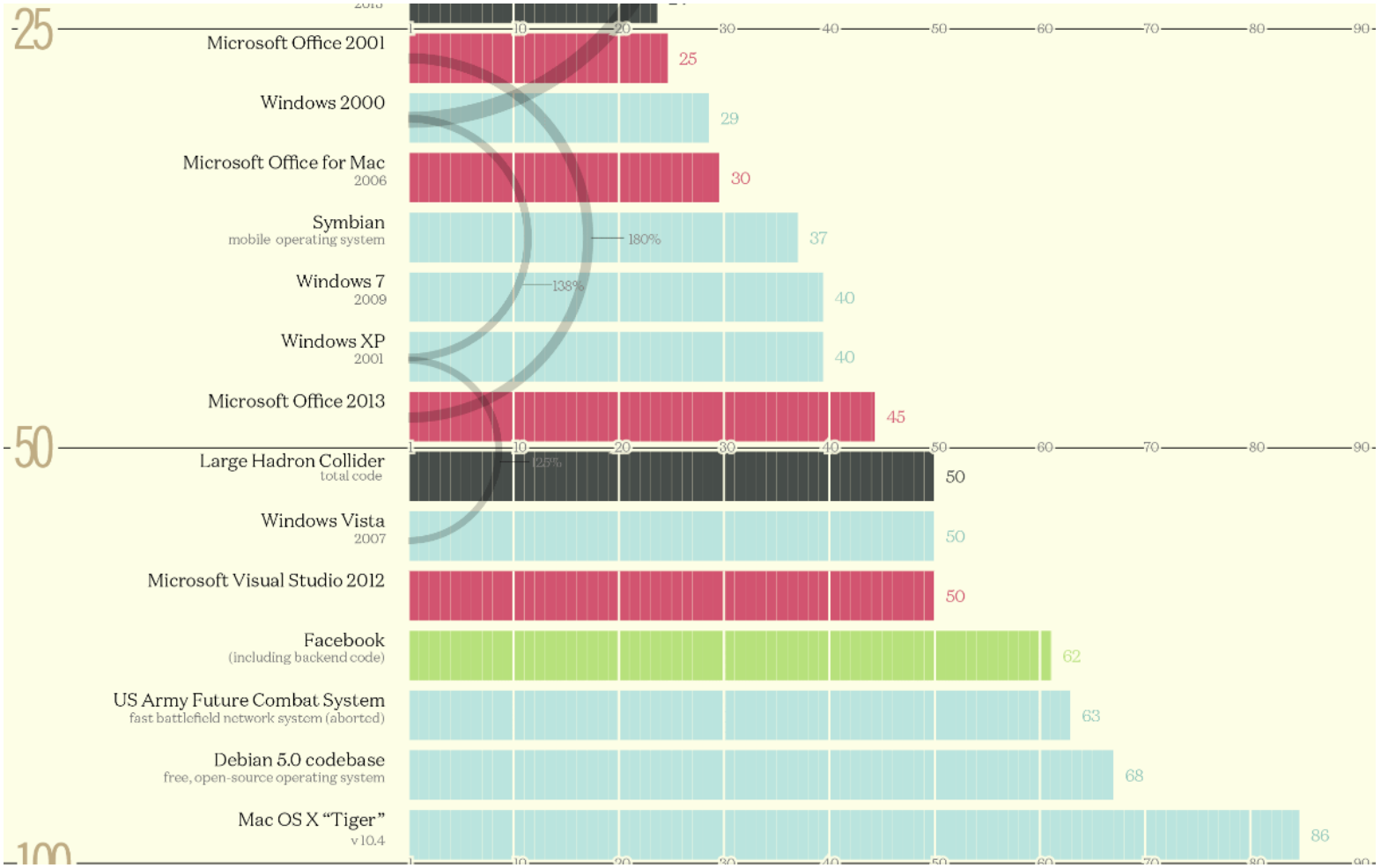
假设1人年生产一万条指令，那么是否4000人生产一年，或400人生产10年就能完成任务？答案是否定的。一万条指令的复杂度决不仅仅是100条指令复杂度的100倍。

Ship Date	Product	Dev Team Size	Test Team Size	Lines of code (LoC)
Jul-93	NT 1.0 (released as 3.1)	200	140	4-5 million
Sep-94	NT 2.0 (released as 3.5)	450	230	7-8 million
May-95	NT 3.0 (released as 3.51)	450	325	9-10 million
Jul-96	NT 4.0 (released as 4.0)	800	700	11-12 million
Dec-99	NT 5.0 (Windows 2000)	1,400	1,700	29+ million
Oct-01	NT 5.1 (Windows XP)	1,800	2,200	40 million
Apr-03	NT 5.2 (Windows Server 2003)	2,000	2,400	50 million

Codebases Million of lines of code

https://informationisbeautiful.net/visualizations/million-lines-of-code/





软件开发中存在的突出问题

软件开发是一个**复杂的系统工程**，其突出问题贯穿需求、技术、团队、管理、安全、产业等多个维度，且随着技术迭代（如AI、信创、全球化）呈现出新旧问题交织、矛盾升级的特征。

- **需求与业务对齐问题**：从“模糊”到“脱节”的恶性循环
- **技术与架构问题**：从“技术债务”到“历史包袱”的累积
- **团队协作与沟通问题**：“沟通”与“流程”的鸿沟
- **安全与合规问题**：从“事后补救”到“事前预防”的转变
- **产业生态问题**：从“规模扩张”到“质量提升”的转型之痛
- **2026年最新趋势**：从“技术驱动”到“生态驱动”的矛盾升级

要解决这些问题，不仅需要技术手段（如重构、自动化测试），更需要管理理念的转变（如敏捷开发、DevOps文化），以及对工程纪律的坚守。

是否存在银弹？



- Brooks, F. P.. *No Silver Bullet: Essence and Accidents of Software Engineering*. IEEE Computer, 1987, 20(4): 10-19.
- Brooks warm us that certain of the difficulties of building software systems derive from the essential nature of software, hence are unlikely to be overcome by any single breakthrough.

Fred Brooks has famously observed that four properties of software, taken together, differentiate it from other kinds of engineering artifacts (Brooks 1995). These four properties are

1. complexity
2. conformity
3. changeability
4. invisibility

作为IBM 360机操作系统项目负责人，总结项目教训时说：

……正像一只逃亡的野兽落到泥潭中做垂死的挣扎，越是挣扎，陷得越深。最后无法逃脱灭顶的灾难，……程序设计工作正像这样一个泥潭，……一批批程序员被迫在泥潭中拼命挣扎，……谁也没有料到问题竟会陷入这样的困境……。

软件工程知识构成

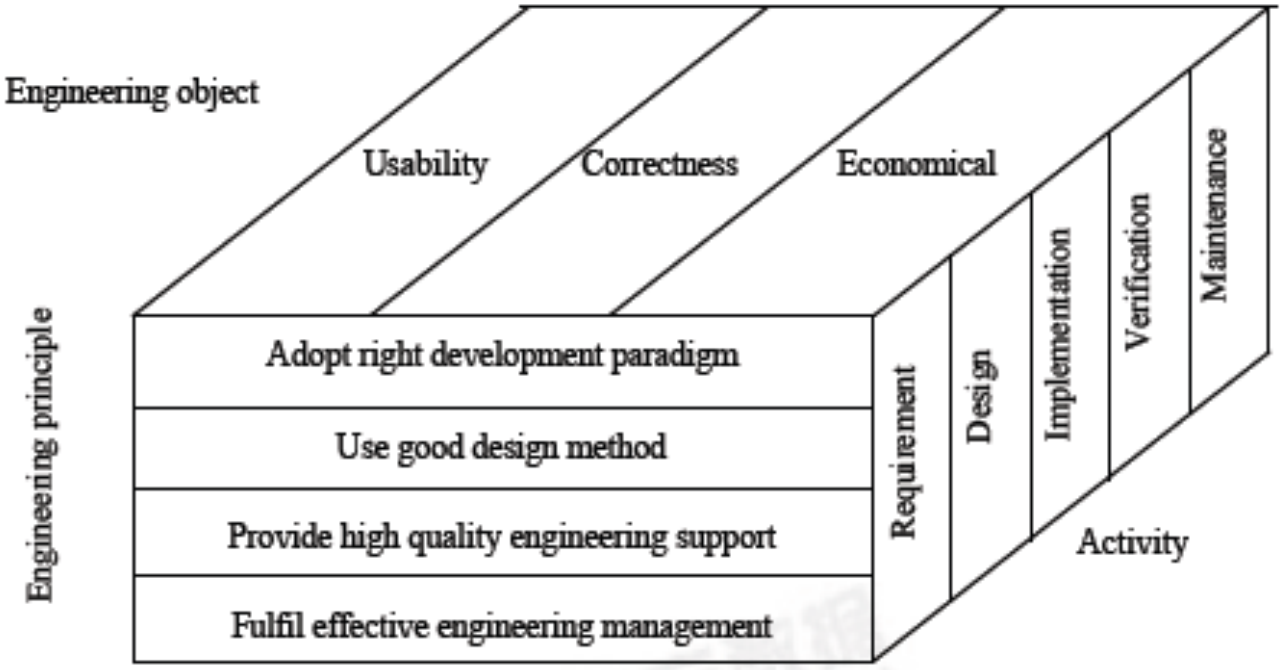


Fig.2 Software engineering framework

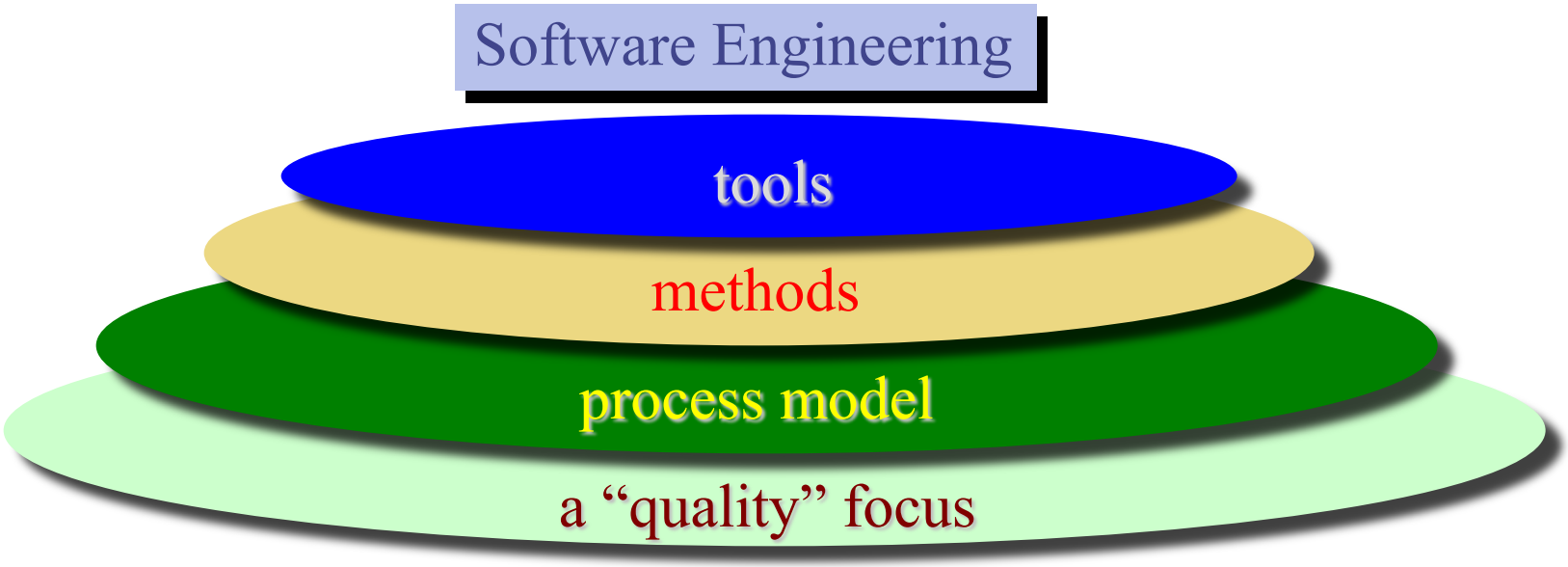
软件工程的三个分支：

- 软件开发技术
- 软件项目管理技术
- 软件质量管理技术

软件工程框架：

- 软件工程的目标
- 软件工程原则
- 软件工程活动

软件工程3要素



软件工程3要素：

① 方法：为软件开发提供“如何做”的技术。

② 工具：为软件工程方法提供自动或半点自动的软件支撑环境。

③ 过程：将软件工程的方法和工具综合起来以达到合理、及时地进行计算机软件开发的目的。

软件工程的四条基本原则：

- 必须认识软件**需求的变动性**，并采取措施来保证结果产品能忠实地满足用户需求。
- 采用**合适的设计方法**。在软件设计中，通常要考虑软件的模块化、抽象与信息隐蔽、局部化、一致性以及适应性等特征。合适的设计方法有助于这些特征的实现，以达到软件工程的目标。
- 软件工程项目的质量与经济开销直接取决于对它所提供的**支撑的质量和效用**。
- 有效的软件工程只有在对**软件过程进行有效管理**的情况下才能实现。

软件工程的四条基本原则：

- 必须认识软件**需求的变动性**，并采取措施来保证结果产品能忠实地满足用户需求。
- 采用**合适的设计方法**。在软件设计中，通常要考虑软件的模块化、抽象与信息隐蔽、局部化、一致性以及适应性等特征。合适的设计方法有助于这些特征的实现，以达到软件工程的目标。
- 软件工程项目的质量与经济开销直接取决于对它所提供的**支撑的质量和效用**。
- 有效的软件工程只有在对**软件过程进行有效管理**的情况下才能实现。

软件工程的基本原理（Barry. Boehm）

① 用分阶段的生命周期计划严格管理

② 坚持进行阶段评审

③ 实行严格的产品控制

④ 采用现代程序设计技术

⑤ 结果应能清楚地审查

⑥ 开发小组的人员应该少而精

⑦ 承认不断改进软件工程实践的必要性

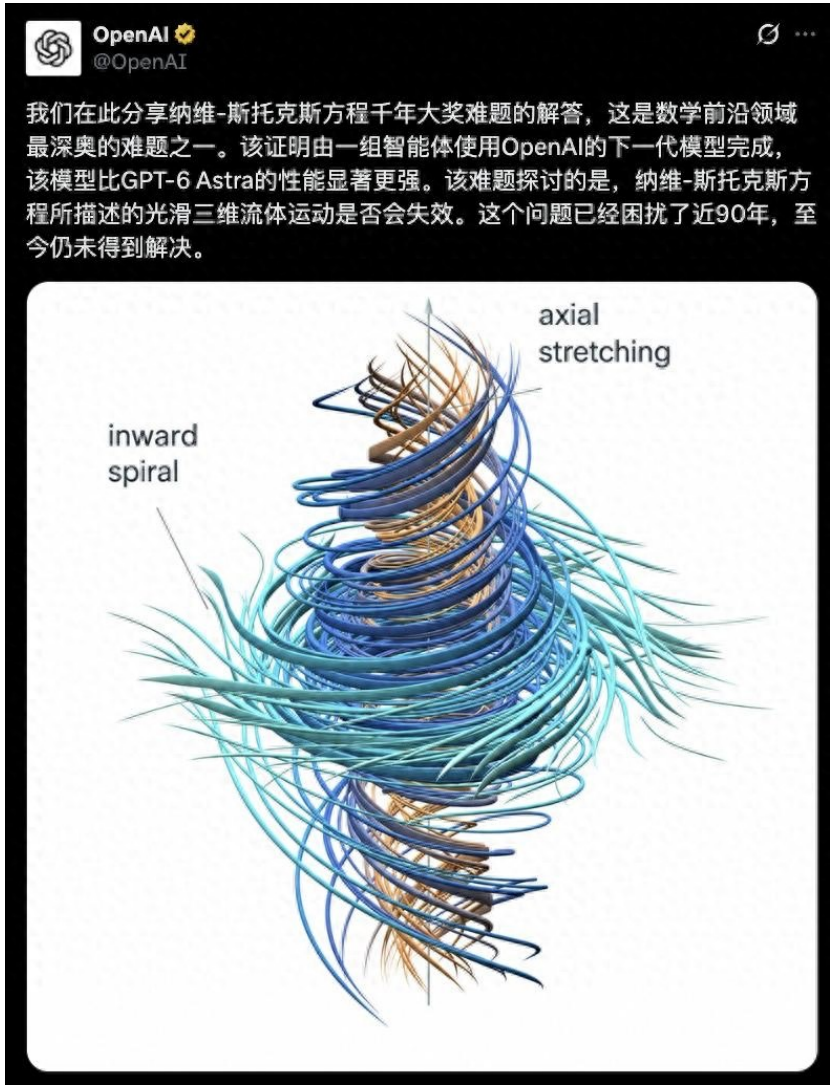


巴利·玻姆 Barry W. Boehm (born 1935) is an American software engineer, Distinguished Professor of Computer Science, Industrial and Systems Engineering; the TRW Professor of Software Engineering; and Founding Director of the Center for Systems and Software Engineering at the University of Southern California. He is known for his many contributions to the area of software engineering.

第1讲　软件与软件工程

1.3 AI驱动的软件开发

【今日头条】难评，OpenAI刚官宣的NS方程大突破，被数学家手撕了……



- NS方程是悬赏100万美元的七大千禧年难题之一
- 为了做出这道题，OpenAI直接摇来约**1万个Agent**，并行工作88小时，发出270万条消息，消耗了约1300亿输出Token。
- OpenAI还透露，这次实验花费了数百万美元。

【算法闲谈】太魔幻！协和脑外科医生去上班，让AI在后台跑了16小时：22年数学猜想据称迎来证明



- 克劳齐猜想（**Crouzeix's conjecture**）——一个矩阵多项式的“拉伸能力”‖$p(A)$‖，至多不会超过该多项式在其矩阵数值域上最大模值的**2**倍。
- **金杉木（Shanmu Jin），中国医学科学院北京协和医院，神经外科**——本科读的是地质学，后来攻读医学博士，现在是神经外科的博士后
- 康奈尔大学、华盛顿大学的数学家，以及在2004年提出该猜想的法国数学家**米歇尔·克劳齐（Michel Crouzeix）**本人参与了核查

早期软件工程体现的特征

- **产品化：**交付符合质量标准的组件、构件和系统。
- **过程决定结果：**流程质量决定产品质量，一环扣一环，相信良好的过程产生良好的产品，关注过程胜过关注人，也非常关注过程评估和过程改进，如CMM是其典型代表。
- **阶段性明确，**需求评审通过了，才能开始设计；设计评审通过了，才能开始实施（编程），编程结束了再进行测试等，如瀑布模型是其典型代表。
- **责任明确：**角色定义清楚，分工细致。
- **文档化：**强调规范的文档，大量的文档模板。
- **计划性强：**完整的计划，严格控制变更。
- **以架构设计为中心：**借鉴建筑工程的框架结构设计，在软件工程上表现为以架构设计为中心。
- **软件生产线：**现代制造业流水线的启发——软件工厂思想。
- **注重项目管理：**围绕项目开展管理工作，包括风险预防、里程碑控制和关键路径法等。
- **质量管理：**以顾客为中心的全面质量管理。
- **缺陷预防：**预防为主，检验为辅。

当代软件工程的特征

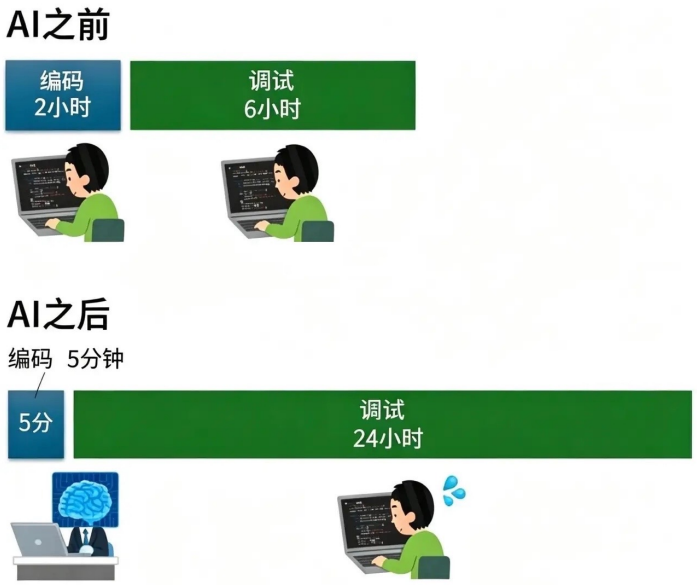
- **SaaS（软件即服务）：**软件更多是以一种服务存在；
- **以人为本：**个体与协作胜于流程和工具，充分发挥个人和团队的创造性和潜力；
- **拥抱变化：**敏捷开发或轻量级过程，加速迭代，以不变应万变。
- **持续性：**阶段性不明确，持续构建、持续集成、持续测试、持续交付，以时间换空间，消除市场风险；
- **融合：**强调测试开发融合，开发与运维融合，推崇全栈工程师等。
- **真正把用户放在第一位：**用户、产品经理尽可能参与团队研发过程中，注重用户体验，千人千面；
- **强调价值交付：**只做对用户有价值的事情，加速价值流的流动；
- **知识管理：**将软件工程纳入知识管理的范畴。
- 史诗般故事、用户故事、站立会议，让软件研发工作更有趣、更健康。
- ……

对软件的新认识

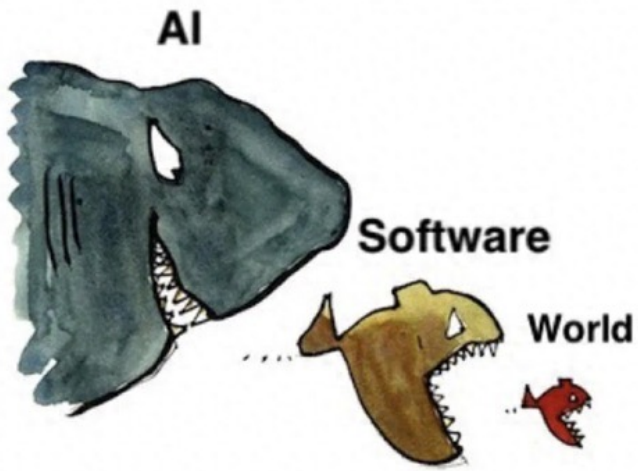
- 软件不再只是代码，而是复杂的**系统化工程**
- 代码从“最终产物”变为“**中间表示**”
- 软件从“静态制品”变为“**动态生命体**”，即软件是持续演化的，不是一次性交付品
- 软件边界从“确定逻辑”扩展到“**概率空间**”
- 软件价值从“功能实现”转向“**体验编排**”
- 软件质量必须 “**左移**”，从源头控制
- 软件**复杂度是最大敌人**，必须主动管理
- 软件安全从“周边防御”转向“**零信任与内生安全**”
- 软件的构建：从"手工作坊"到"**人机协同的生态系统**"
- ……

软件工程新情况

- **核心范式转移**：从“AI辅助编程”到“智能体编程 (Agentic Coding)”
- **“杰文斯悖论”**显现：效率提升反而导致需求爆炸
- **云原生与架构**：全面普及 + 深度演进
- **工程门槛不降反升**：从“语法熟练度”到“系统判断力”
- **开发民主化**：低代码 / 无代码 + 公民开发者
- **DevSecOps**：安全左移 + 默认安全
- **技术栈与生态**：国产化 + 全栈自主
- **安全与合规的新战场**
- **人才结构的断层与重组**



AI对软件工程的影响



- 以“**效率提升、流程重构、角色转型**”为核心，改变软件工程的开发模式与价值逻辑。
- AI驱动的**软件工程价值逻辑重构**，本质上是技术范式跃迁引发的经济学价值重分配过程。
- 核心在于通过**边际成本坍塌**和创造力解放，重塑软件价值创造、分配与捕获的**底层逻辑**。

下棋容易，洗碗难。

生活在一个既充满危险又充满希望的时代既是悲剧，又是喜剧。

真相残酷：代码已成“白菜价”，软件工程才是你的保命符！



Anthropic最新2026趋势报告：人类最大一次编程革命势不可挡



趋势1：软件开发生命周期，正在剧变
趋势2：多智能体协作
趋势3：构建完整系统
趋势4：人机智能协作
趋势5：扩展新场景
趋势6：重塑开发经济学
趋势7：非技术岗应用
趋势8：安全优先架构

“代码将死，逻辑永存”

Software was eaten by AI.

对软件和软件工程的再认识

- 软件的本质困难在于复杂性、一致性、易变性和不可见性，它涉及问题空间理解、需求取舍、平台能力、成本管理和系统演进，这些靠数据学习无法实现。
- 软件工程的核心复杂性不在于写代码，而在于对问题域的理解、权衡取舍、成本控制与长期演进的预判。软件行业的核心竞争力从来不是写代码，而是解决复杂问题。
- 软件工程不是“写代码的流程”，而是人类对抗复杂性、缺陷、不确定性和组织失误的防御体系。



第1讲　软件与软件工程

1.4 软件生存期模型

软件开发生命周期

软件开发生命周期（**SDLC**）是软件行业用来设计、开发和测试高质量软件的过程。

SDLC的目标是生产高质量的软件，满足或超过客户期望，在时间和成本估算内完成。



软件生存周期（Software Life Cycle）

GB/T 11457-2006 《信息技术——软件工程术语》

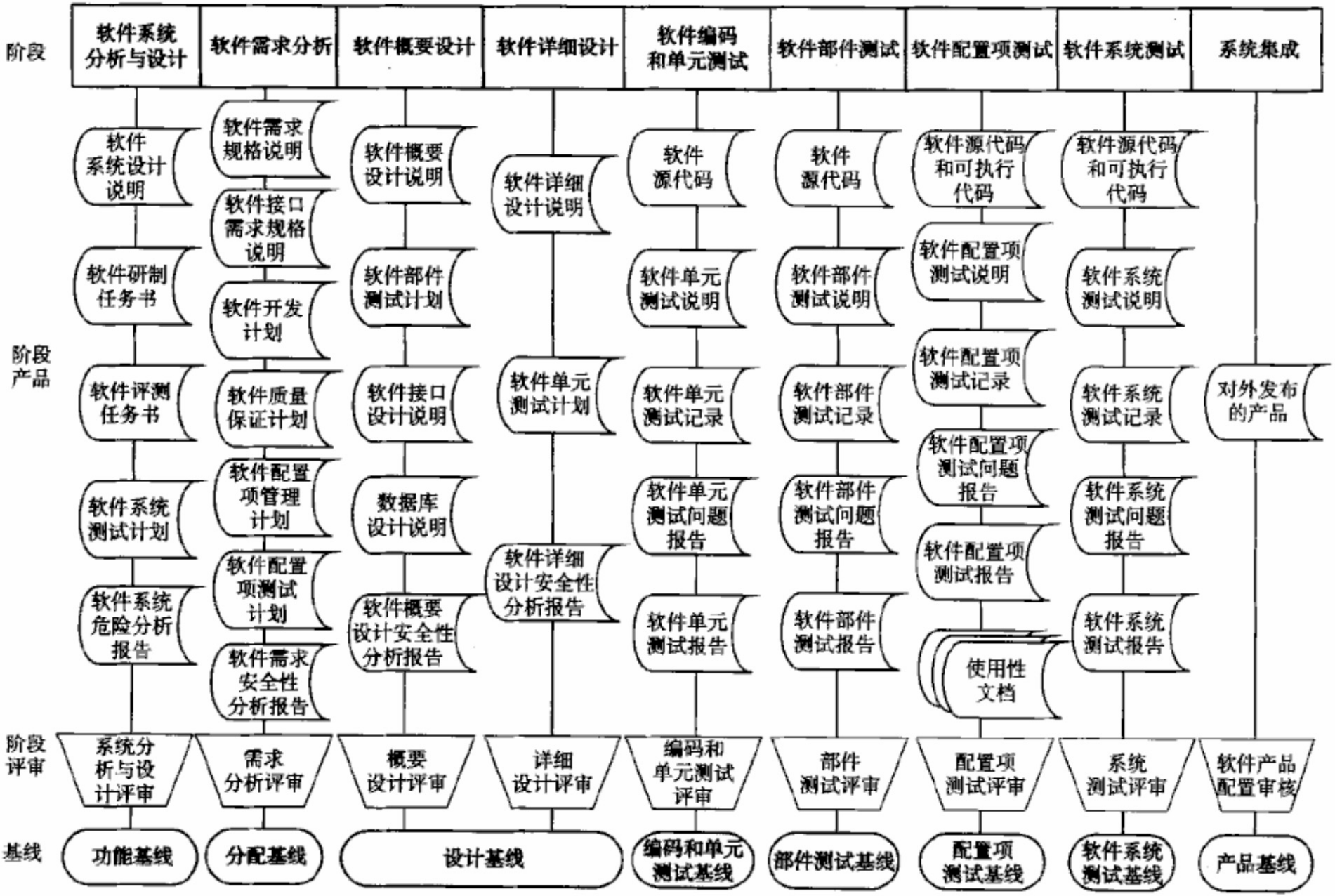
- 软件产品从构思开始至软件不再可用结束时的时间周期。
- 软件生存周期典型的地包括：需要阶段、设计阶段、实现阶段、测试阶段、安装和验收阶段、操作和维护阶段，有时还包括退役阶段。

ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes（系统和软件工程——软件生命周期过程）将软件过程概括为4类：

- 协商过程
- 组织项目启用过程
- 技术管理过程
- 技术过程

软件生命周期过程

协商过程	采购过程
	供应过程
组织项目启用过程	生命周期模型管理过程
	基础设施管理过程
	投资组合过程
	人力资源管理过程
	质量管理过程
技术管理过程	项目规划过程
	项目评估和控制过程
	决策管理过程
	风险管理过程
	配置管理过程
	信息管理过程
	测量过程
	质量保证过程
技术过程	业务和任务分析过程
	干系人需求和需求定义过程
	系统/软件需求定义过程
	架构定义过程
	设计定义过程
	系统分析过程
	实现过程
	集成过程
	验证过程
	转移过程
	确认过程
	运行过程
	维护过程
	处置过程


阶段
软件系统分析与设计
软件需求分析
软件概要设计
软件详细设计
软件编码和单元测试
软件部件测试
软件配置项测试
软件系统测试
系统集成
阶段产品
软件系统设计说明
软件研制任务书
软件评测任务书
软件系统测试计划
软件系统危险分析报告
软件需求规格说明
软件接口需求规格说明
软件开发计划
软件质量保证计划
软件配置项管理计划
软件配置项测试计划
软件需求安全性分析报告
软件概要设计说明
软件部件测试计划
软件接口设计说明
数据库设计说明
软件概要设计安全性分析报告
软件详细设计说明
软件单元测试计划
软件详细设计安全性分析报告
软件源代码
软件单元测试说明
软件单元测试记录
软件单元测试问题报告
软件单元测试报告
软件源代码
软件部件测试说明
软件部件测试记录
软件部件测试问题报告
软件部件测试报告
软件源代码和可执行代码
软件配置项测试说明
软件配置项测试记录
软件配置项测试问题报告
软件配置项测试报告
使用性文档
软件源代码和可执行代码
软件系统测试说明
软件系统测试记录
软件系统测试问题报告
软件系统测试报告
对外发布的产品
阶段评审
系统分析与设计评审
需求分析评审
概要设计评审
详细设计评审
编码和单元测试评审
部件测试评审
配置项测试评审
系统测试评审
软件产品配置审核
基线
功能基线
分配基线
设计基线
编码和单元测试基线
部件测试基线
配置项测试基线
软件系统测试基线
产品基线

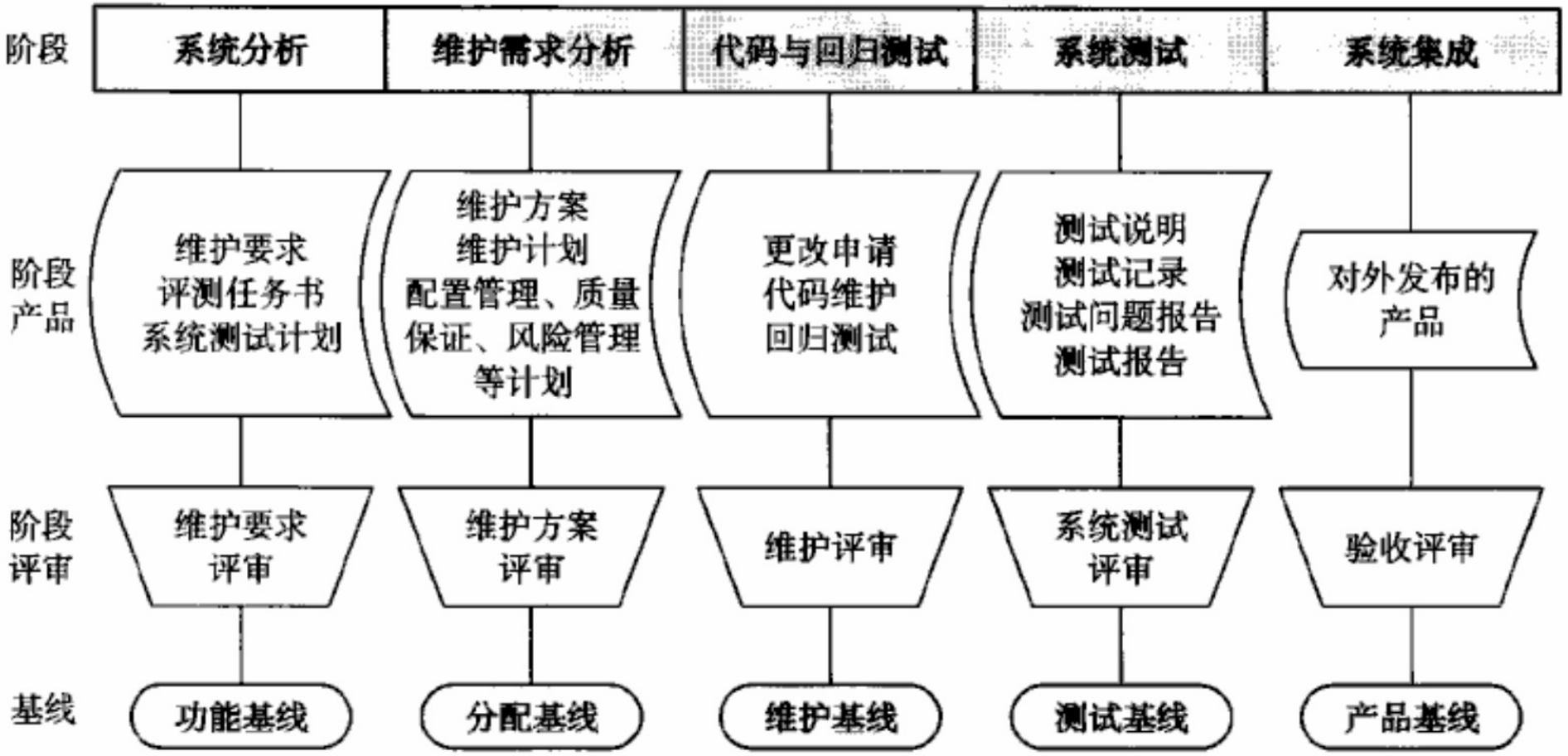

阶段
系统分析
维护需求分析
代码与回归测试
系统测试
系统集成
阶段产品
维护要求
评测任务书
系统测试计划
维护方案
维护计划
配置管理、质量保证、风险管理等计划
更改申请
代码维护
回归测试
测试说明
测试记录
测试问题报告
测试报告
对外发布的产品
阶段评审
维护要求评审
维护方案评审
维护评审
系统测试评审
验收评审
基线
功能基线
分配基线
维护基线
测试基线
产品基线

图 2　软件维护过程

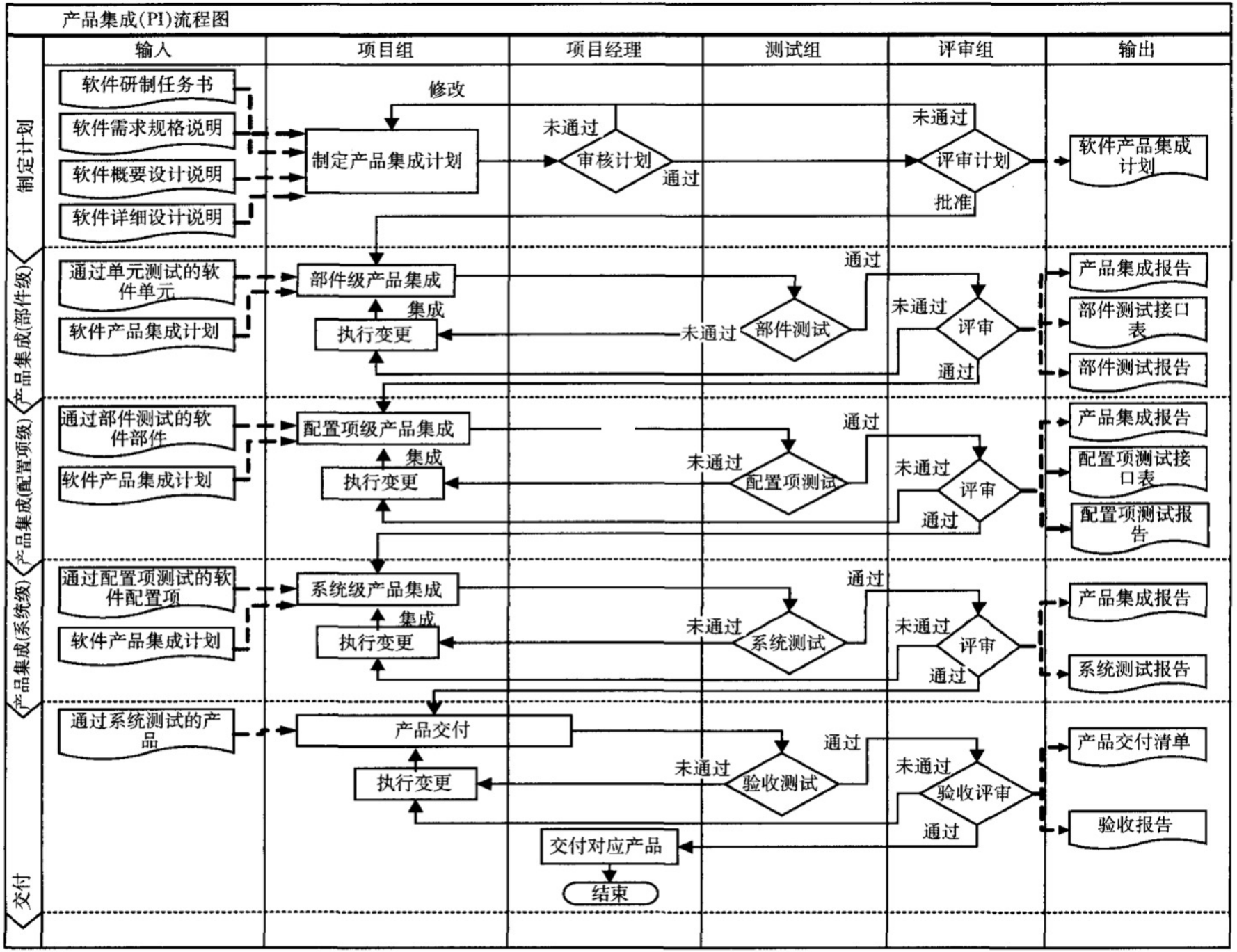


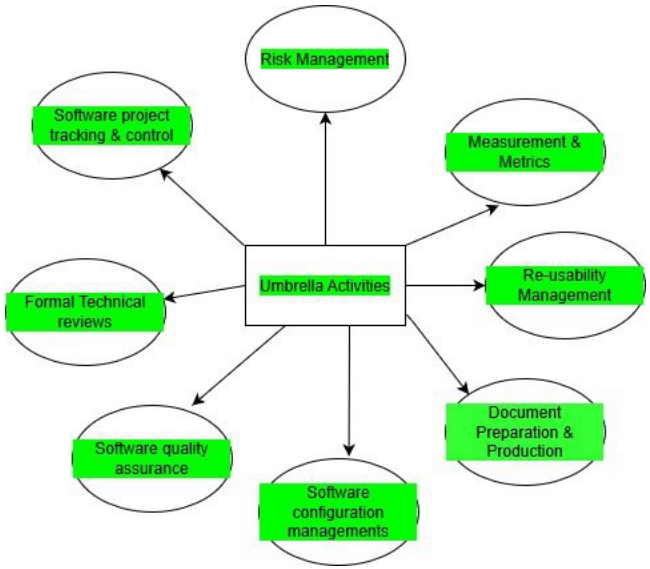
图 产品集成过程域实施流程

软件生存期主要任务

1. 问题定义
   - 这个阶段必须回答的关键问题是：“要解决的问题是什么”。
2. 可行性研究
   - 这个阶段要回答的关键问题是：“上一个阶段所确定的问题是否有行得通的解决办法”。
3. 需求分析
   - 这个阶段的任务仍然不是具体地解决客户的问题，而是准确地回答“目标系统必须做什么”这个问题。
   - 这个阶段的另外一项重要任务，是用正式文档准确地记录对目标系统的需求，这份文档通常称为规格说明(specification)。
4. 概要设计
   - 这个阶段的基本任务是，概括地回答“怎样实现目标系统?”这个问题。
     - 首先，应该设计出实现目标系统的几种可能的方案。
     - 概要设计的另一项主要任务就是设计软件的体系结构，也就是确定程序由哪些模块组成以及模块间的关系。
5. 详细设计
   - 这个阶段的任务是设计出程序的详细规格说明。就是把解法具体化，也就是回答“应该怎样具体地实现这个系统”这个关键问题。
6. 编码和单元测试
   - 这个阶段的关键任务是写出正确的容易理解、容易维护的程序模块。
7. 综合测试
   - 这个阶段的关键任务是通过各种类型的测试（及相应的调试）使软件达到预定的要求。
8. 软件维护
   - 维护阶段的关键任务是，通过各种必要的维护活动使系统持久地满足用户的需求。
   - 通常有四类维护活动：
     - 改正性维护，也就是诊断和改正在使用过程中发现的软件错误；
     - 适应性维护，即修改软件以适应环境的变化；
     - 完善性维护，即根据用户的要求改进或扩充软件使它更完善；
     - 预防性维护，即修改软件为将来的维护活动预先做准备。

保护性活动

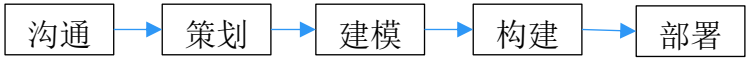
- 保护性活动（Umbrella activity)是软件开发团队为了维护完整开发任务的进度、质量、更改和风险而遵循的一系列步骤或过程。
- 保护性活动包括的任务：
  - 软件项目跟踪和控制
  - 正式技术审查
  - 软件质量保证
  - SCM或软件配置管理
  - 文件编制和制作
  - 重用性管理
  - 测量和指标
  - 风险管理



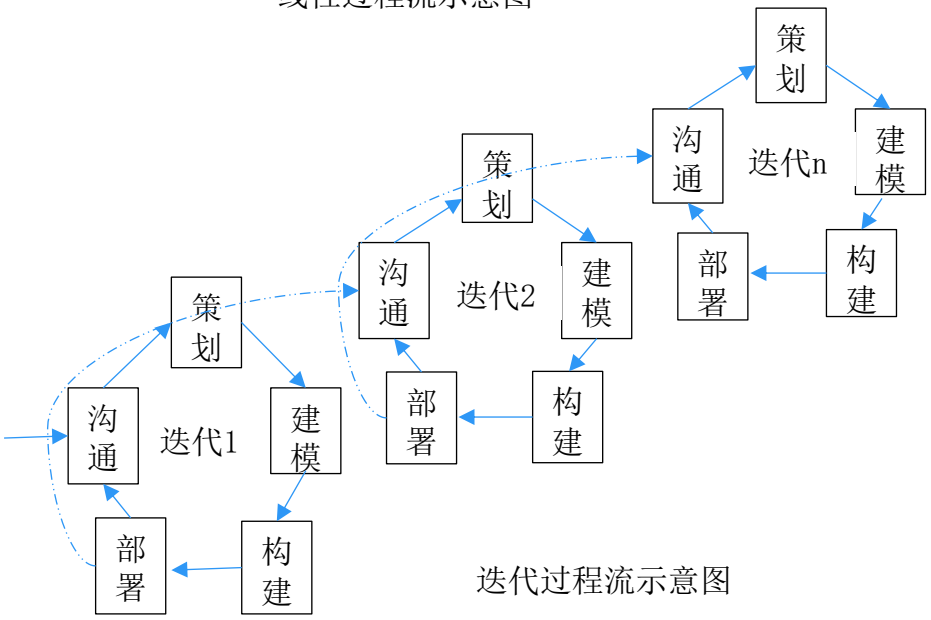
过程流

过程流（process flow）描述了在执行顺序和时间上如何组织框架中的活动任务，执行顺序和时间的安排直接影响到里程碑、基线和交付物的设置。

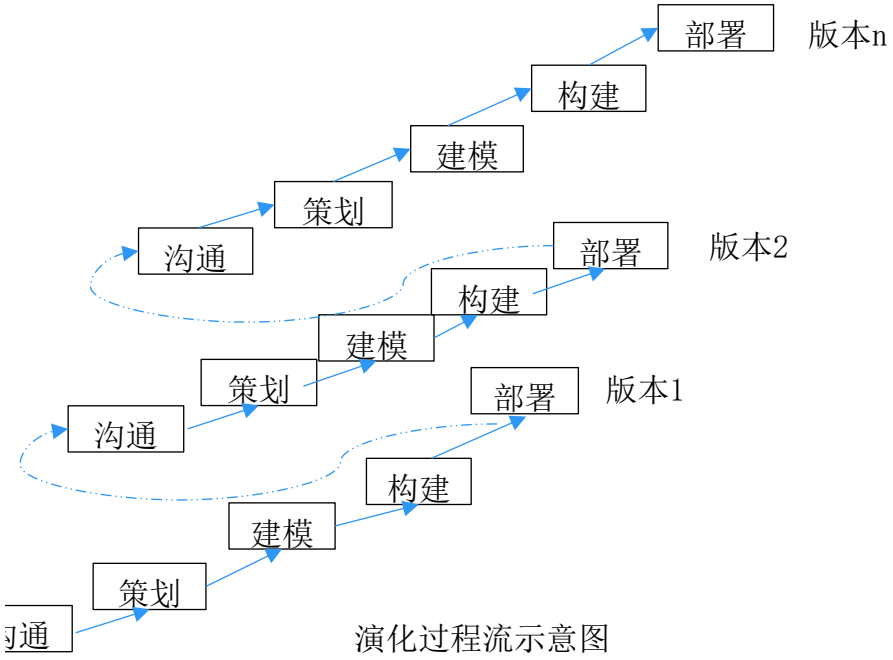
- 线性过程流（linear process flow）
- 迭代过程流（iterative process flow）
- 演化过程流（evolutionary process flow）



线性过程流示意图



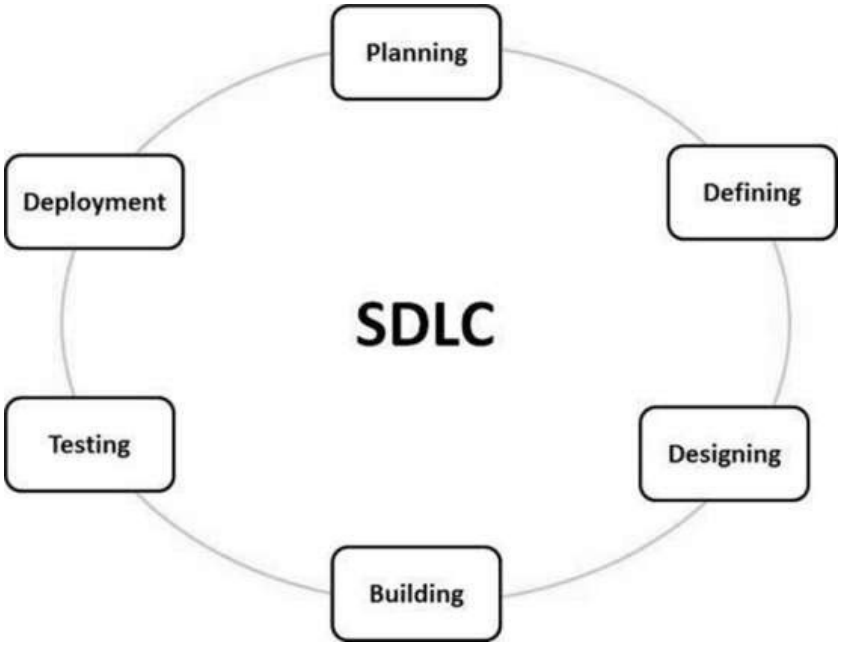
迭代过程流示意图



演化过程流示意图

主要的软件生存期模型(life cycle model)

- 瀑布模型
- 原型模型
- 快速应用开发（RAD）模型
- 螺旋模型
- 增量模型和迭代模型
- 构件组装模型
- 并发模型
- 测试驱动（TDD）型的软件开发
- Rational的RUP和UML
- 形式化描述技术FDT
- 敏捷方法（XP）

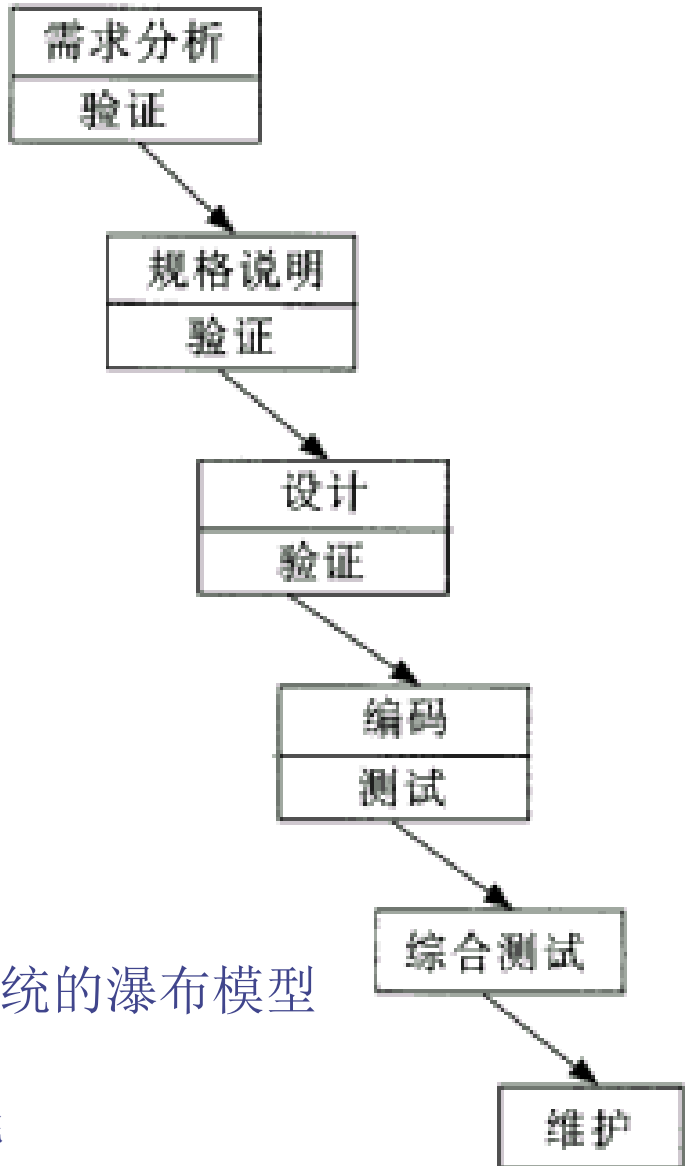


软件生存期模型意义：

- 软件开发模型是软件开发全部过程、活动和任务的结构框架。
- 软件开发模型能清晰、直观地表达软件开发全过程，明确规定了要完成的主要活动和任务，用来作为软件项目开发的基础。

瀑布模型（Waterfall Model）

- 1970年，Winston Royce提出软件开发的瀑布模型。
- 在20世纪80年代之前，瀑布模型一直是唯一被广泛采用的生命周期模型，现在它仍然是软件工程中应用最广泛的过程模型之一。
- 按照传统的瀑布模型来开发软件，有如下几个特点：
  (1) 阶段间具有顺序性和依赖性
  (2) 质量保证的观点
  (3) 推迟实现的观点

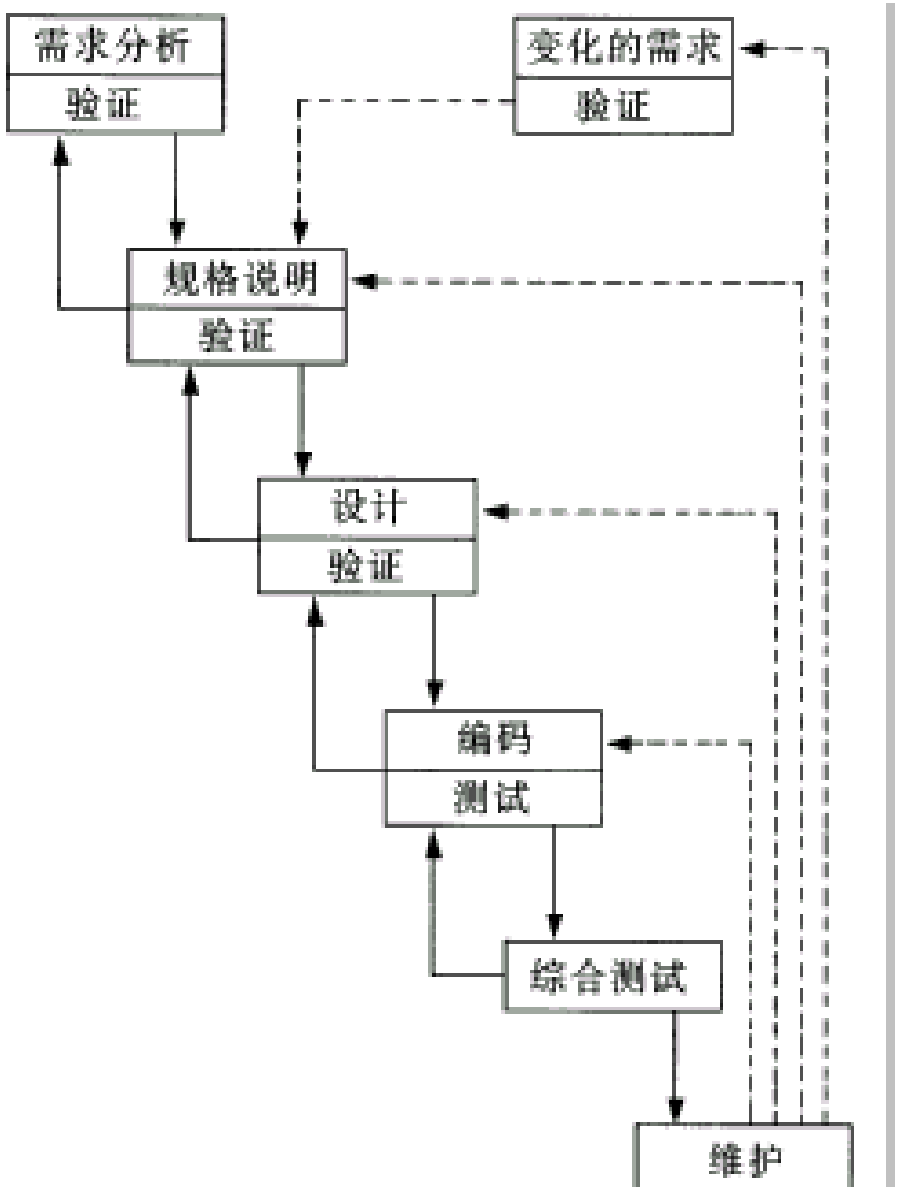


传统的瀑布模型

实际的瀑布模型

瀑布型的特点：

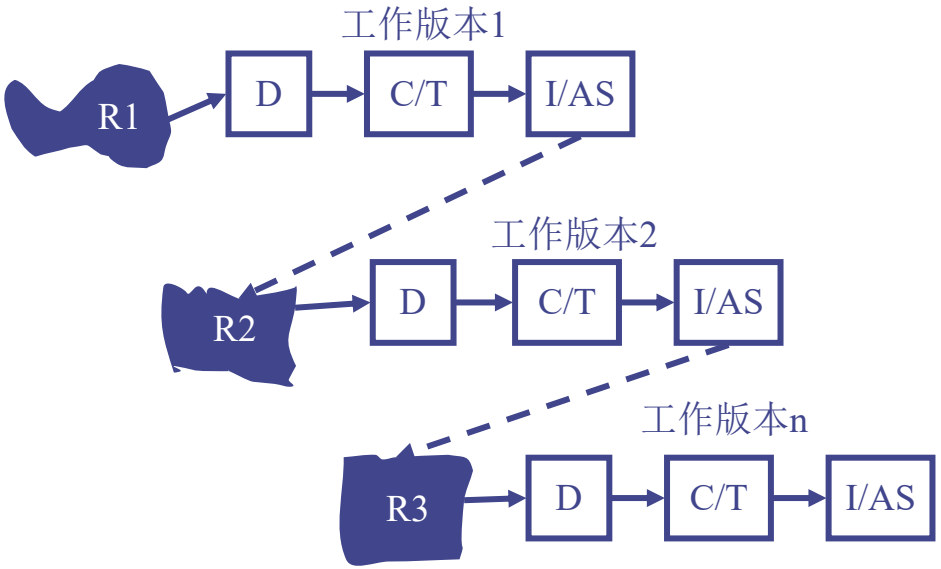
- 清楚地区分逻辑设计与物理设计，尽可能推迟程序的物理实现。
- 每个阶段都必须完成规定的文档，没有交出合格的文档就是没有完成该阶段的任务。
- 每个阶段结束前都要对所完成的文档进行评审，以便尽早发现问题，改正错误。



演化模型（evolutionary model）

- 演化模型主要针对事先不能完整定义需求的软件项目，此模型可以减少风险。
- 为了探索可行性和弄清需求，作第一次试验开发，以取得有效的反馈信息，来支持软件的最终设计和实现。
- 第一次试验开发出来的软件称为原型（prototype）。原型法是一种从基本需求出发，快速构建系统的原型，通过原型确认需求，改进原型，最终达到建立系统的目的。

- 演化模型的多种形式：
  - 丢弃型
  - 样品型
  - 渐增式演化型

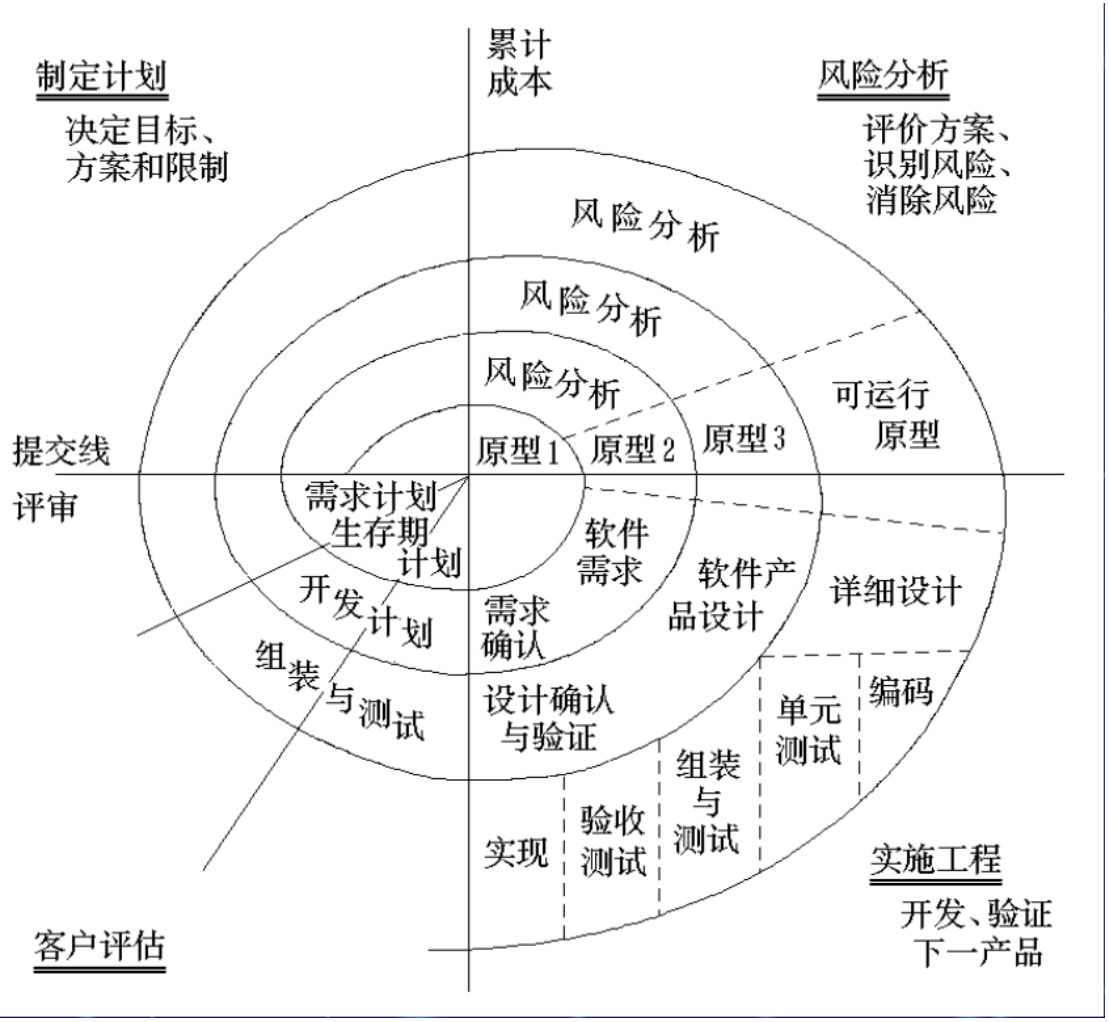


R：需求 C/T：编码/测试 D：设计 I/AS：安装和验收支持

演化模型示例

螺旋模型（Spiral Model）

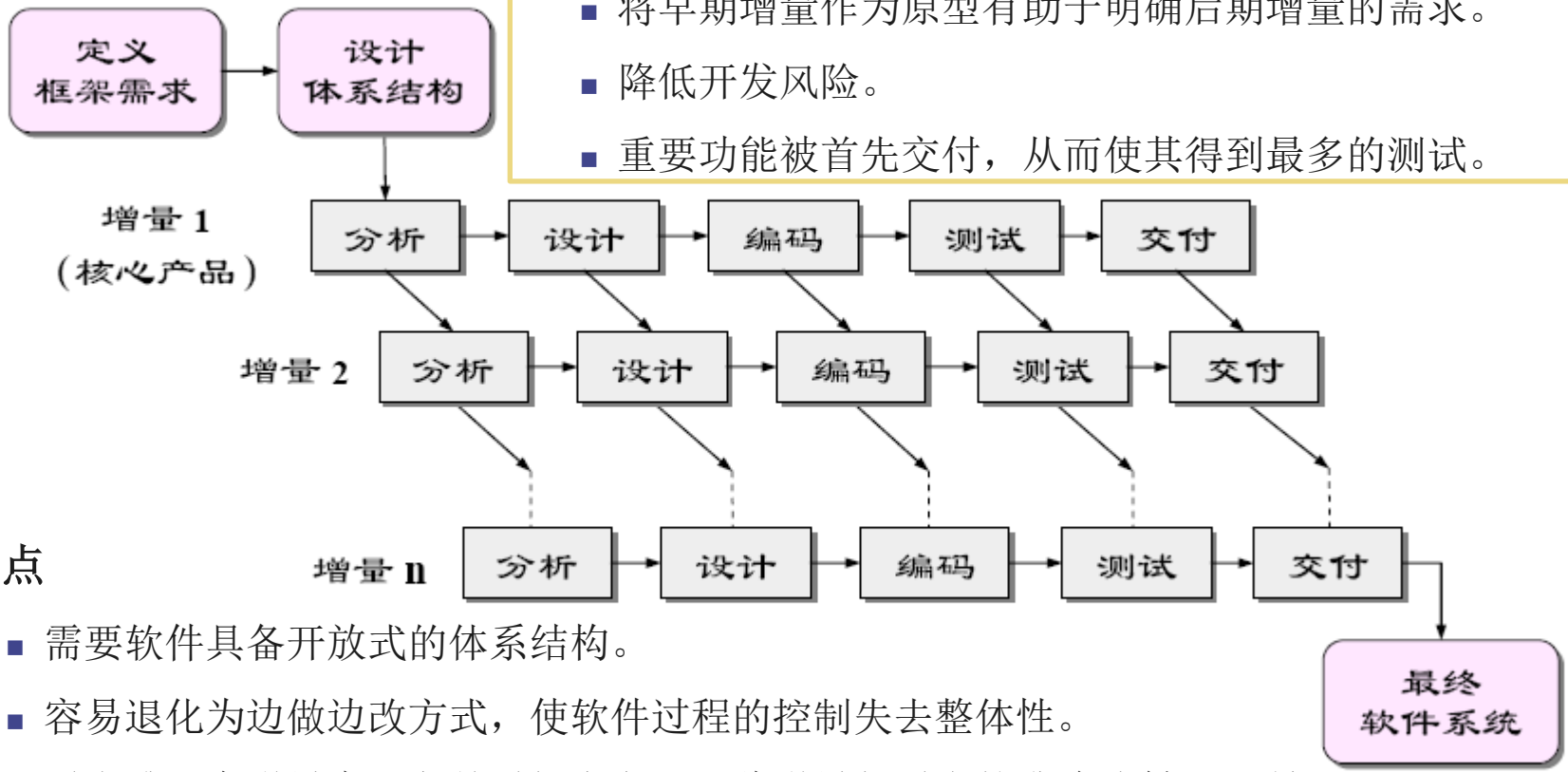
- 螺旋模型是瀑布模型与演化模型结合，并增加两者所忽略的风险分析。
- 螺旋模型通常用来指导大型软件项目开发，它将开发划分为：
  - 制定计划
  - 风险分析
  - 实施开发
  - 客户评估



增量模型

优点

- 整个产品被分解成若干个构件逐步交付，用户可以不断地看到所开发软件的可运行中间版本。
- 将早期增量作为原型有助于明确后期增量的需求。
- 降低开发风险。
- 重要功能被首先交付，从而使其得到最多的测试。

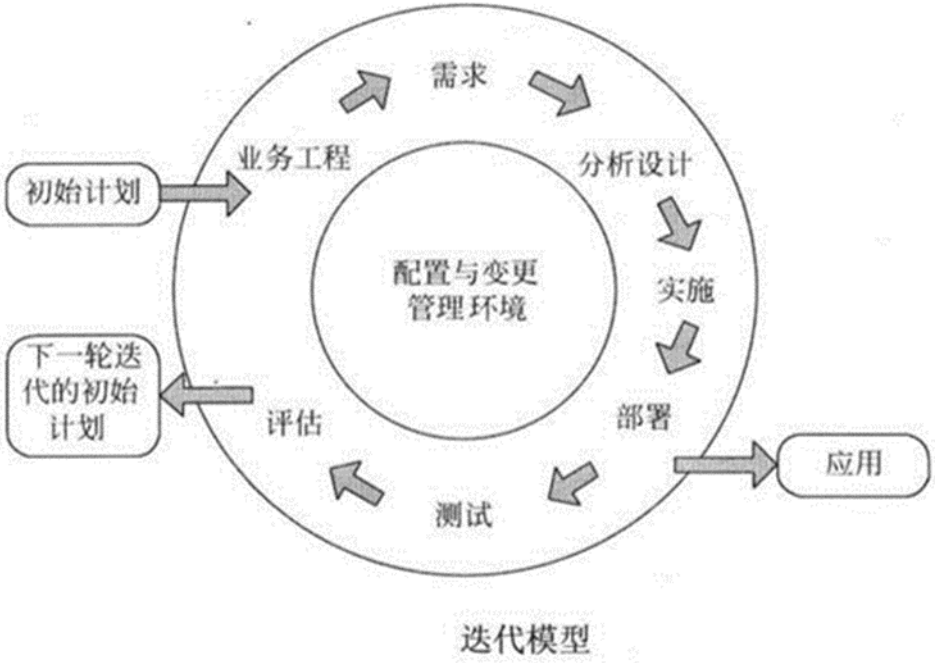


缺点

- 需要软件具备开放式的体系结构。
- 容易退化为边做边改方式，使软件过程的控制失去整体性。
- 需求难以在增量实现之前详细定义，因此增量与需求的准确映射以及所有增量的有效集成可能会比较困难。

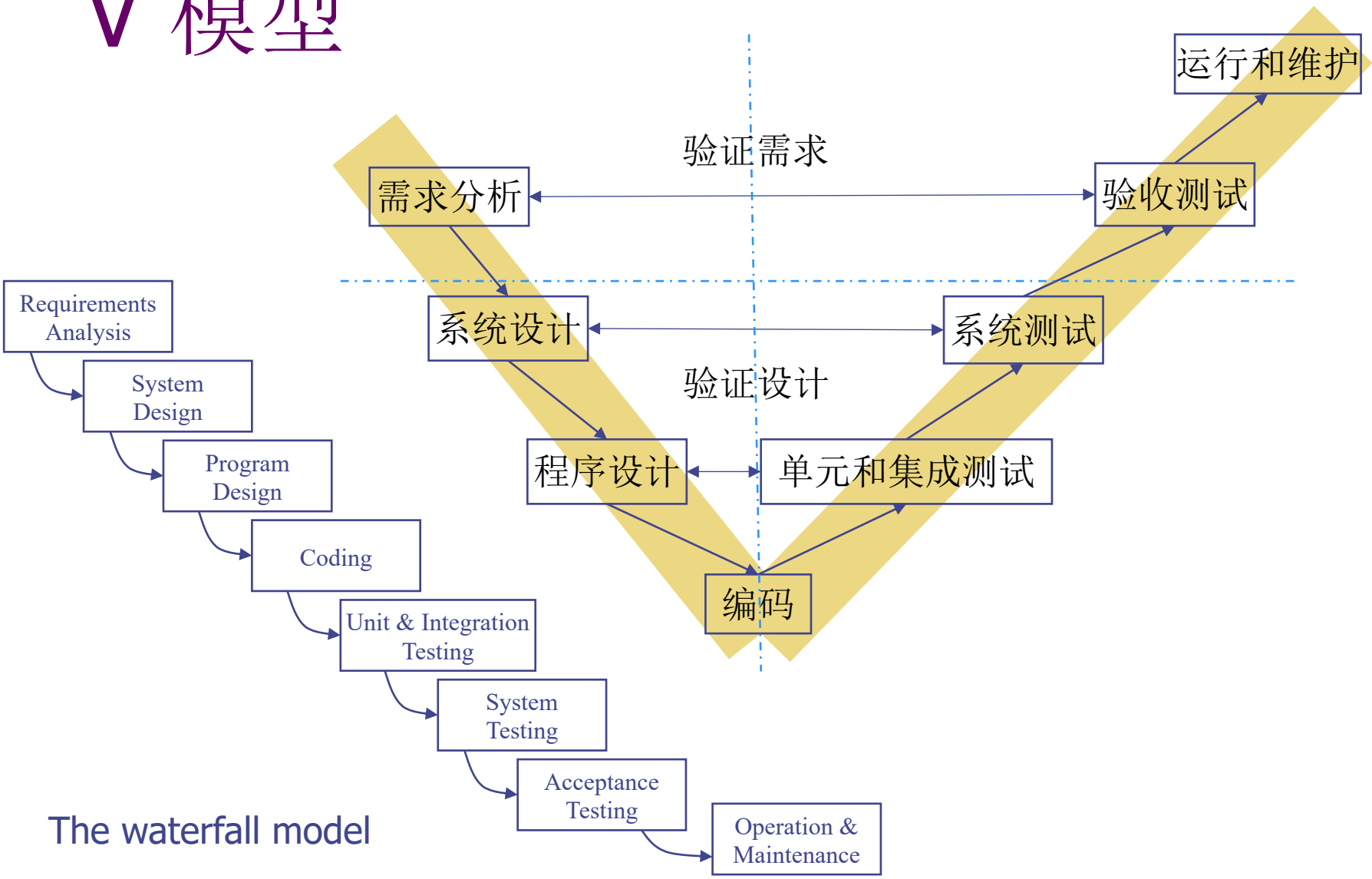
迭代模型

- 迭代模型是瀑布模型的缩小与循环，每次迭代都会产生一个可以发布的软件版本，该版本是最终产品的一个子集。
- 迭代模型的优点：
  - 用户反馈时间短；
  - 演进式推进可以降低软件项目的风险
- 采用迭代模型的项目，往往需求变更频繁，易导致软件体结构频繁变动，对项目的管理、质量控制、开发团队有非常高的要求。



迭代模型

V 模型



The waterfall model

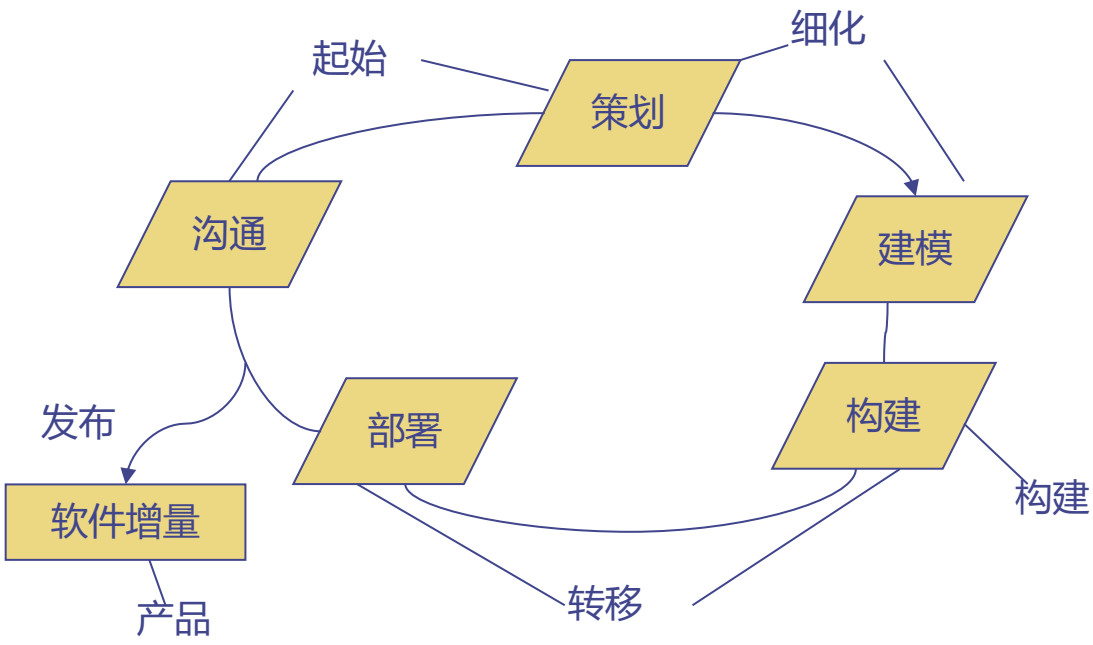
V模型的特点

- V模型是瀑布模型的变体，编码（coding）构成了V点，左边是分析和设计，右边是测试和维护，它表达了测试与分析和设计的对应相关，该模型更明确地说明了隐藏在瀑布描绘中的一些迭代和返工。
- 编码形成了V点，左边是分析和设计，右边是测试和维护。
- V型关键：
  - V的左侧与V右侧的关联意味着，如果在验证和确认期间发现问题，可以重新执行V左侧，以在重新执行右侧的测试步骤之前修复和改进需求、设计和代码。
  - 该模型使瀑布描述中隐藏的一些迭代和返工更加明确。

统一过程（RUP）

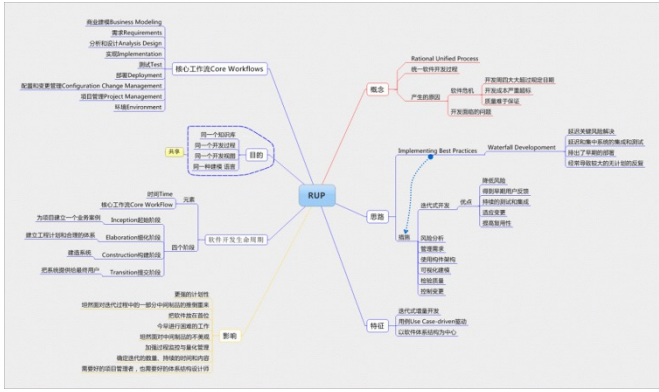
- RUP 是一种通用的、面向对象的方法，它是Rational公司（现在是IBM的一部分）在20世纪90年代初开发的。
- RUP是一种迭代增量式的软件开发方法论，它强调将系统开发过程分为一系列的迭代周期，每个周期都包括了需求分析、设计、编码、测试和部署等活动。
- RUP的主要特点和思想包括：
  - 迭代增量开发
  - 用例驱动
  - 架构中心
  - 风险驱动
  - 集成开发环境（IDE）
  - 需求管理：

统一过程（RUP）



1. **初始阶段**：建立一个业务案例或理由，确定核心需求，初步风险评估，原型。
2. **细化阶段**：建立架构，建立综合计划，交付的是系统行为模型（系统环境、场景、领域模型），以及基于领域模型的基线产品构想、开发计划、评估标准产品发布描述，其他还有用户手册、测试计划等。
3. **构造阶段**：编程，产品包括可执行代码、系统和用户手册、部署计划、质量保证结果等工作产品。
4. **移交阶段**：将产品发布给用户进行测试评价，并收集用户的意见，之后再次进行迭代修改产品使之完善。
5. **发布阶段**：已完成的所有制品。

RUP最佳实践



(1) 迭代式开发

- 在软件开发的早期阶段完全、准确的捕获用户的需求几乎是不可能的。
- 需求在整个软件开发工程中经常会改变。
- 迭代式开发允许在每次迭代过程中需求可能有变化，通过不断细化来加深对问题的理解。
- 迭代式开发不仅可以降低项目的风险，而且每个迭代过程都以可执行版本结束，可以鼓舞开发人员。

(2) 管理需求

- 确定需求是一个连续的过程，开发人员在开发系统之前不可能完全详细的说明一个系统的真正需求。
- 用例和脚本的使用已被证明是捕获功能性需求的有效方法。

(3) 体系结构

- 组件使重用成为可能，系统可以由组件组成。
- 基于独立的、可替换的、模块化组件的体系结构有助于降低管理复杂性，提高重用率。

(4) 可视化建模

- 用UML对软件系统建立可视化模型，帮助人们提供管理软件复杂性的能力。

(5) 验证软件质量

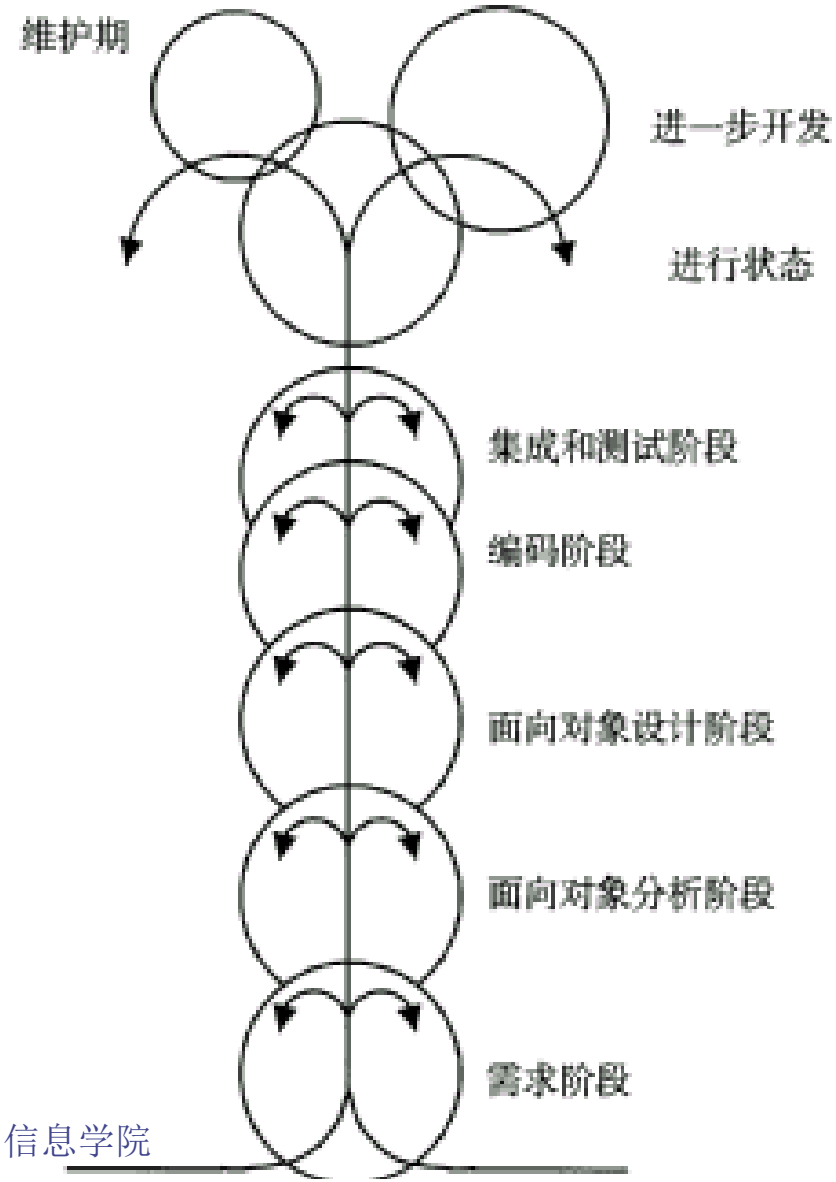
- 软件质量评估不是事后进行或单独小组进行的分离活动，而是内置于过程中的活动，这样可以及早发现软件中的缺陷。

(6) 控制软件变更

- 迭代式开发中如果没有严格的控制和协调，整个软件开发过程很快就陷入混乱之中。
- 通过软件开发过程中的制品，隔离来自其他工作空间的变更，以此为每个开发人员建立安全的工作空间。

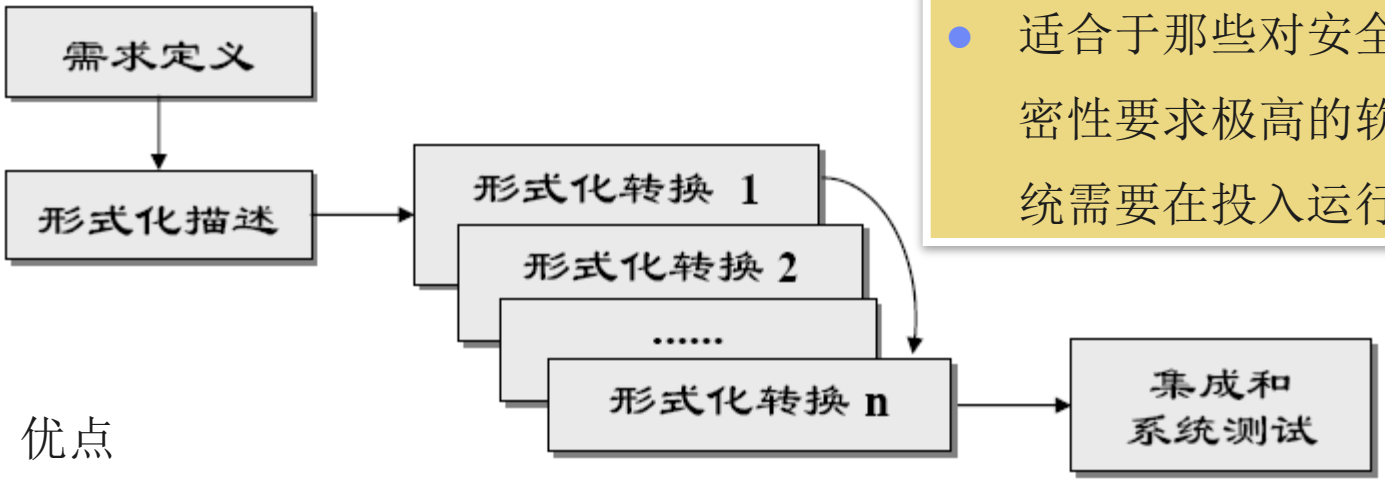
喷泉模型（Water Fountain Model）

- 喷泉模型主要用于采用对象技术的软件开发项目，“喷泉”这个词体现了面向对象软件开发过程迭代和无缝的特性。
- 为避免使用喷泉模型开发软件时开发过程过分无序，应该把一个线性过程(例如，快速原型模型或图中的中心垂线)作为总目标。但是，同时也应该记住，面向对象范型本身要求经常对开发活动进行迭代或求精。



形式化方法模型

- 形式化方法模型是采用形式化的数学方法将系统描述转换成可执行的程序。
- 适合于那些对安全性、可靠性和保密性要求极高的软件系统，这些系统需要在投入运行前进行验证。



- 优点
  - 由于数学方法具有严密性和准确性，形式化方法开发过程所交付的软件系统具有较少的缺陷和较高的安全性
- 缺点
  - 开发人员需要具备一定技能并经过特殊训练
  - 形式化描述和转换是一项费时费力的工作
  - 现实应用的系统大多数是交互性强的软件，但是这些系统难以用形式化方法进行描述

软件生命周期模型选择问题

经过软件工程多年的经验积累，可以通过已知的项目特点来选择适合的生命周期模型，以此减少项目已知的风险，生命周期模型选择的原则如下：

系统环境特征	级别	适用模型
项目风险级	高	Spiral
需求理解程度	低	Spiral
需求更改频率	高	Iterative/Incremental
产品定义程度	高	Waterfall, RAD
用户参与程度	高	Iterative, RAD
发布日期压力	高	Iterative/Incremental
员工经验	低	Waterfall, Iterative
平台移植	经常	Iterative/Incremental

项目特点	瀑布模型	增量模型	原型+瀑布	迭代
需求规模	中、小	大、中	中、小	大、中、小
需求清晰度	很清晰	较清晰	不清晰	不清晰
客户信息化能力	高	中	中	中、低
客户界面要求	低	中	高	高
行业特点	行业规则简单或已掌握的行业	新行业的应用型项目或已掌握的行业	新行业的应用型项目	新行业的研发型或应用型项目
架构设计能力	有较成熟的架构设计能力	采用了新的架构设计	采用新的架构设计	不能确定的架构设计
技术要求	采用成熟的技术	采用较少的新技术	采用较少的新技术	采用较多的新技术
管理要求	工作产品的控制基线，需要有较高的可见度和可靠性 。	每个增量阶段需要发布可操作产品。	原型阶段，需要控制原型制作的成本。	需要有较高的项目控制能力。每个迭代阶段需要有明确的目标。

软件过程模型选择时需要考虑的主要因素

项目特点	瀑布	增量	原型	螺旋	演化	迭代	统一过程
行业特点	成熟领域	所有行业	新兴行业	所有行业	所有行业	新兴行业或研发型软件	所有行业
需求清晰度	清晰	较清晰	不清晰	不清晰	不清晰	不清晰	不清晰
特性要求（安全性、可靠性等）	高	高、中、低	中、低	高、中	中、低	中、低	中、低
软件复杂度	中、低	高、中	中、低	高、中	中、低	高、中、低	高、中、低
交付期压力	低	高	高、中	低	低	高、中	高、中
技术要求	成熟技术	少量新技术	新技术	新技术	成熟或新技术	新技术	成熟或新技术
项目风险	中、低	中、低	高、中	高	中、低	高、中	高、中
团队经验	不要求	不要求	有相似项目的经验	熟悉相似项目	熟悉相似项目	不要求	不要求
架构重构能力	成熟架构	成熟架构	架构重构	成熟架构	架构重构	架构重构	架构重构
管理要求	基线控制	版本规划	沟通和激励，原型验证	风险管理	过程控制	迭代周期控制	过程控制
用户参与度	低	低	必须参与	必须参与	中	高、中	高、中

选择软件过程模型实质上是选择软件开发的策略，是为软件开发的主要活动提供一套范型，使工程的进展有章可循。

评估软件过程模型的适用性

需要回答的问题如下：

1）模型是否适合团队的规模和他们的技能?

2）模型是否适合项目中使用的技术?

3）模型是否适合客户和利益相关者?

4）模型是否适合项目规模及其复杂度?

5）模型是否能满足企业的商业目标?

6）模型是否能应对项目的风险?

7）产品开发过程中是否会有任何变化?

8）需求是否明确或需求变更频率与模型的特性是否相适应?

9）用户或客户参与软件开发的程度能否满足模型的要求？

10）软件交付期的压力对模型的要求是什么？

11）开发团队的软件过程能力能否适应模型的要求？

12）客户对模型有什么要求？

AI时代的软件开发生命周期

- 软件开发生命周期（SDLC，Software Development Life Cycle）是软件工程领域的一个核心框架， 是一套结构化的流程或方法论，它定义了**将软件从概念转化为可运行产品，并最终交付给用户**所需经历的一系列明确阶段。
- AI-DLC （AI-Driven Development Lifecycle），将传统 SDLC 重构为由 AI 深度参与、持续反馈的人机协作生命周期。
- 核心工作：
  - 将软件交付拆分为**Inception（构想）→ Construction（构建）→ Operations（运营）**三个连续阶段
  - AI 负责生成、分析、诊断和建议，人类负责业务意图、架构取舍、安全合规与最终门禁
  - 强调持续学习与反馈闭环，让需求、代码、测试、运行数据共同反哺下一轮交付

第1讲 软件与软件工程

1.5 软件工程知识体系

SWEBOK® GUIDE V4.0

Public review version (2024 Jan 16th)

TABLE I.1. THE 18 SWEBOK KAS

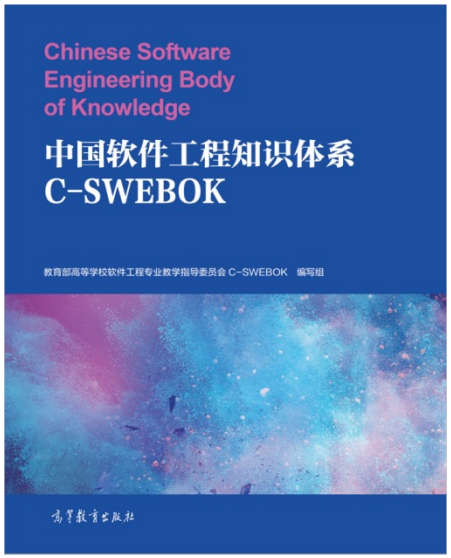
1. Software Requirements
2. Software Architecture
3. Software Design
4. Software Construction
5. Software Testing
6. Software Engineering Operations
7. Software Maintenance
8. Software Configuration Management
9. Software Engineering Management
10. Software Engineering Process
11. Software Engineering Models and Methods
12. Software Quality
13. Software Security
14. Software Engineering Professional Practice
15. Software Engineering Economics
16. Computing Foundations
17. Mathematical Foundations
18. Engineering Foundations

TABLE I.2. RELATED DISCIPLINES

19. Computer Engineering
20. Computer Science
21. Cybersecurity
22. Data Science
23. General Management
24. Information Systems and Technology
25. Mathematics
26. Project Management
27. Quality Management
28. Systems Engineering

- KA: Knowledge Area
- SWEBOK: Software Engineering Body of Knowledge

C-SWEBOK



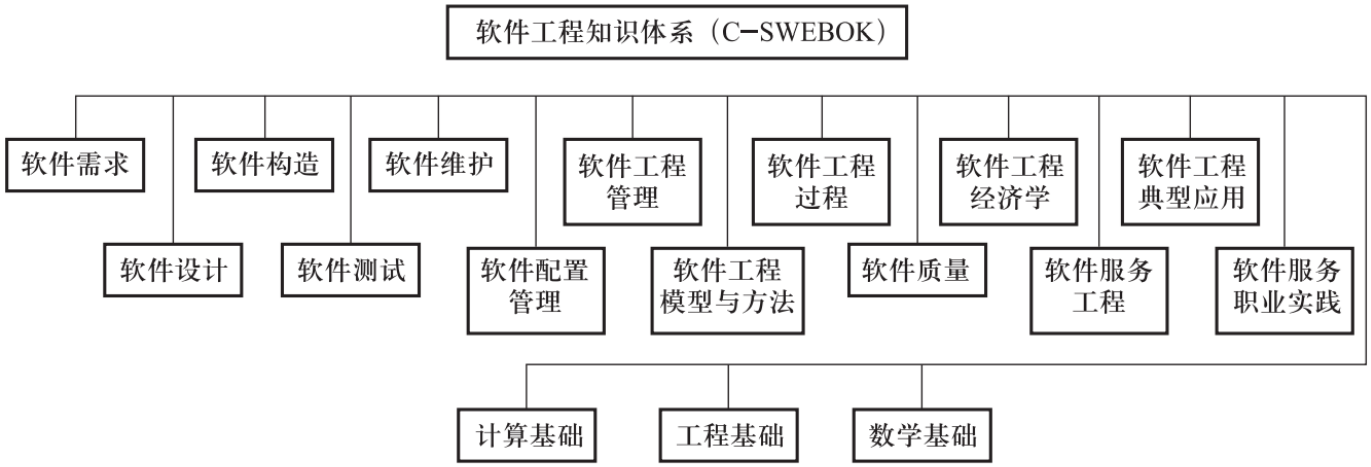


图 3-1　C-SWEBOK 知识体系框架

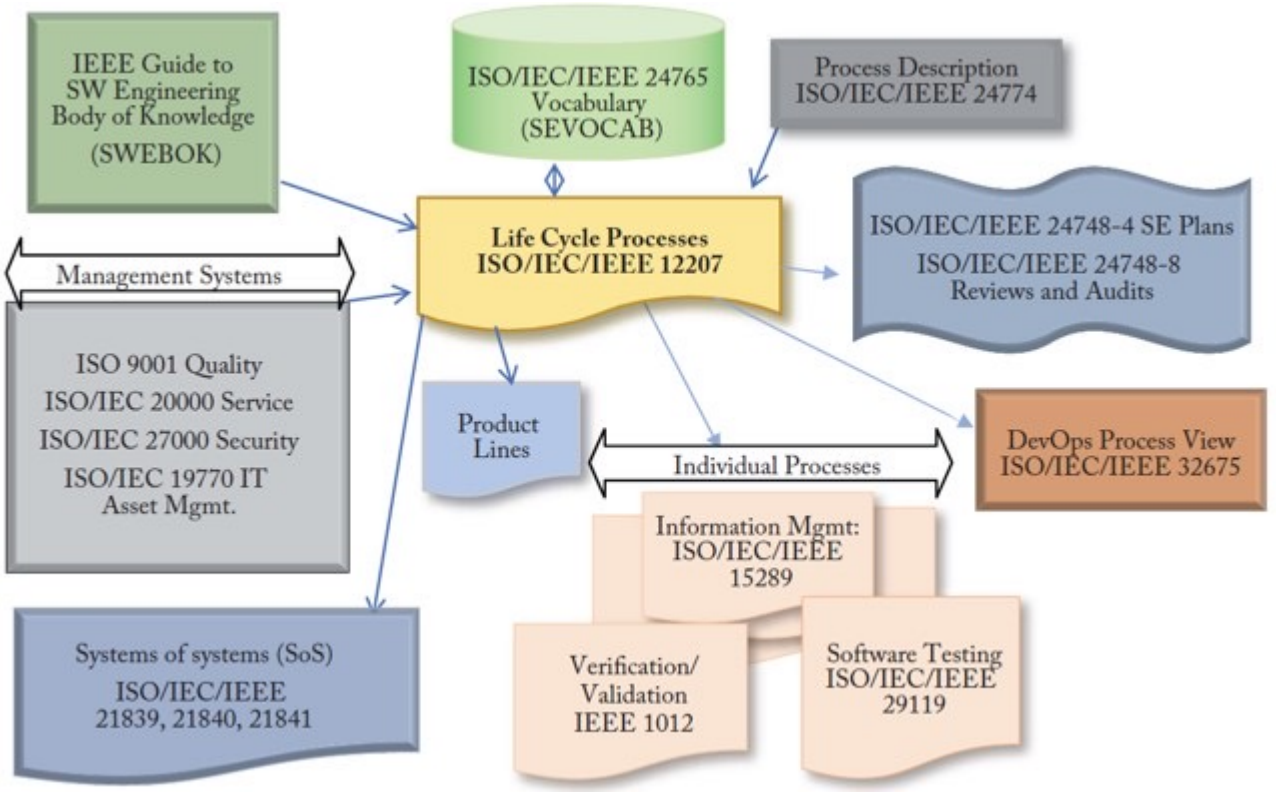
C-SWEBOK在继承SWEBOK 3.0的基础上，结合我国软件工程一级学科建设与软件工程专业教育现状，以及我国高校软件学院的教学实践经验，融入部分中国软件工程教育元素，新增软件服务工程、软件工程典型应用2个知识领域，扩充软件工程职业实践知识领域，以满足我国软件人才培养的需要。

第1讲　软件与软件工程

1.6 软件工程师的职业特点

软件工程专业实践的法律问题

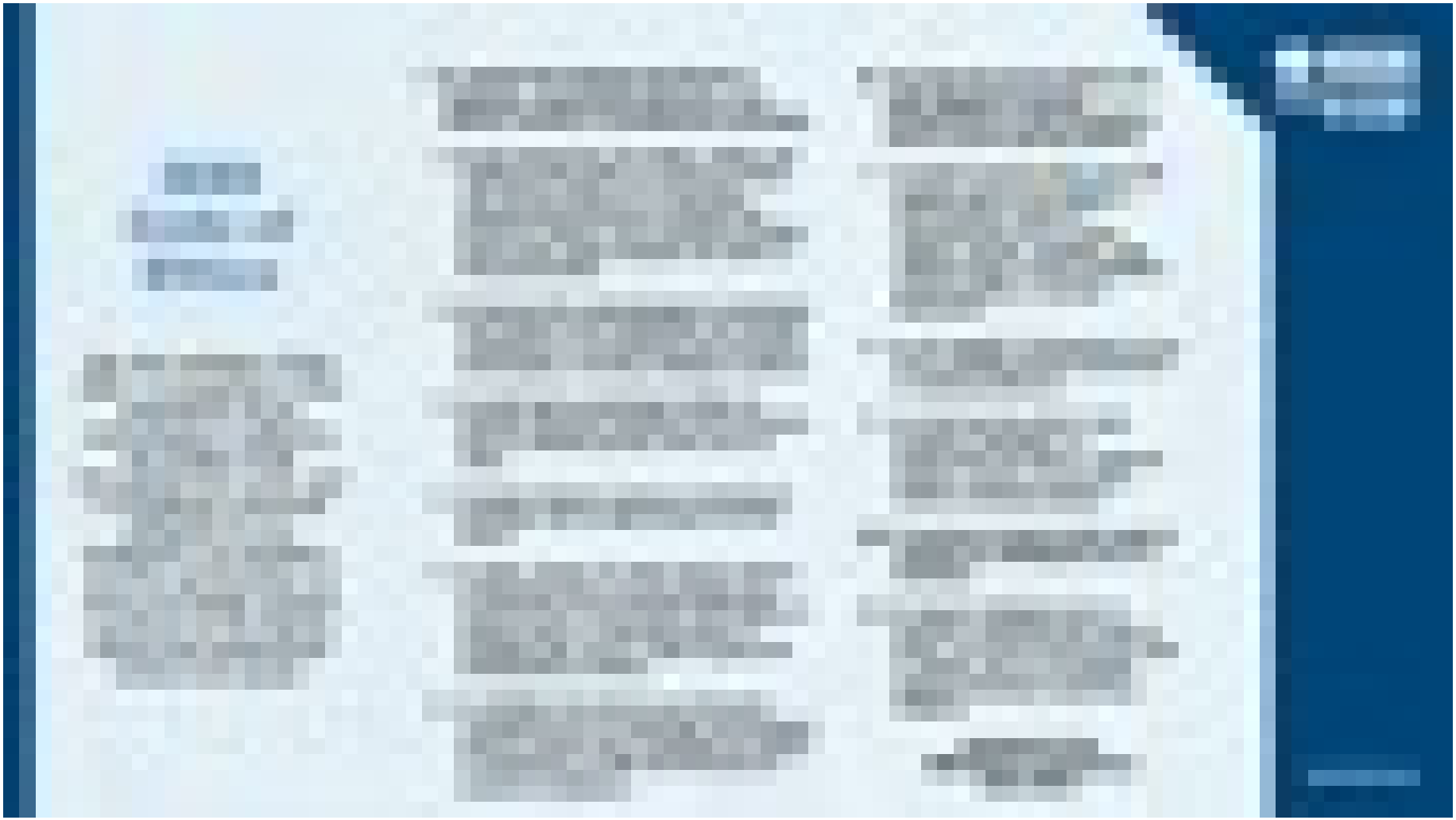
- 标准
- 商标
- 专利
- 版权
- 商业秘密
- 专业责任
- 法律要求
- 贸易合规性
- 网络犯罪
- 数据隐私



软件从业人员需遵守的法律和法规

- 著作权法
  - 软件开发者必须遵守著作权法，确保他们开发的软件没有侵犯他人的知识产权，包括版权、专利和商标等。
- 数据保护法
  - 软件开发者必须遵守数据保护法，确保他们的软件处理用户数据的方式符合相关法规，同时保护用户隐私。
- 反垄断法
  - 软件开发者必须遵守反垄断法，确保他们的软件不会垄断市场或破坏公平竞争。
- 网络安全法
  - 软件开发者必须遵守网络安全法，确保他们的软件没有安全漏洞或后门，防止黑客攻击和数据泄露。
- 计算机软件保护条例
  - 中国针对计算机软件保护制定的法规，规定了软件开发者和用户的权利和义务，保护软件知识产权。
- 信息产业部软件产品保护条例
  - 中国信息产业部发布的软件保护规定，规定了软件开发者和用户的责任和义务，保护软件知识产权和用户权益。

IEEE Code of Ethics



软件工程职业道德规范和实践要求 5.2 版

IEEE-CS 和 ACM 软件工程道德和职业实践联合工作组推荐

软件工程师应履行其实践承诺，使软件的需求分析、规格说明、设计、开发、测试和维护成为一项有益和受人尊敬的职业。为实现他们对公众健康、安全和利益的承诺目标，软件工程师应当坚持以下八项原则：

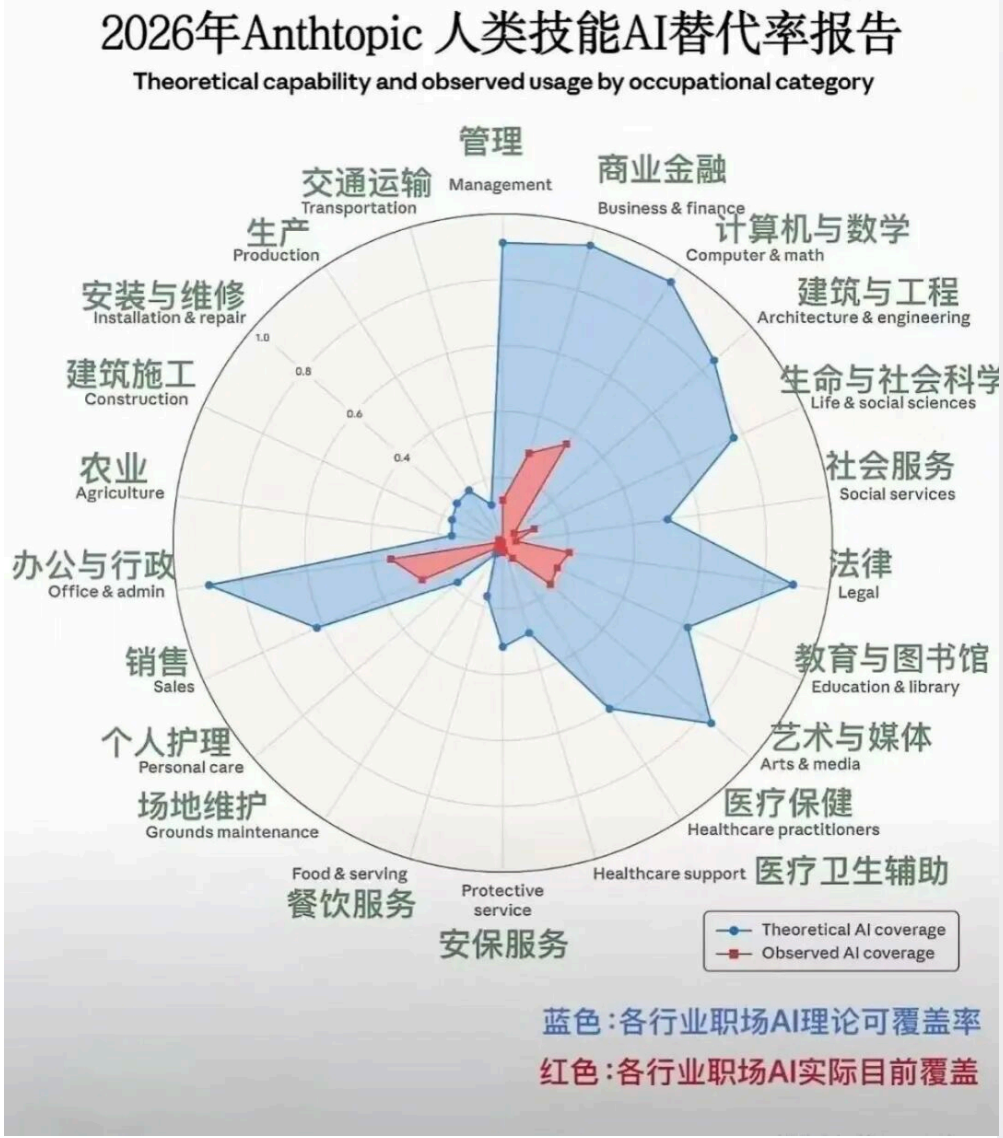
1. 公众 – 软件工程师应当以公众利益为目标；
2. 客户和雇主 – 在保持与公众利益一致的原则下，软件工程师应注意满足客户和雇主的最高利益；
3. 产品 – 软件工程师应当确保他们的产品和相关的改进符合最高的专业标准；
4. 判断 – 软件工程师应当维护他们职业判断的完整性和独立性；
5. 管理 – 软件工程的经理和领导人员应赞成和促进对软件开发和维护合乎道德规范的管理；
6. 专业 – 在与公众利益一致的原则下，软件工程师应当推进其专业的完整性和声誉；
7. 同行 – 软件工程师对其同行应持平等和互助和支持的态度；
8. 自我 – 软件工程师应当参与终生职业实践的学习，并促进合乎道德的职业实践方法。

AI对职业的影响

Anthropic于2026年3月5日发布研究报告《Labor market impacts of AI: A new measure and early evidence》，核心内容包括：

- 提出衡量 AI 对职业替代风险的新指标：“观测暴露度”（Observed Exposure）；
- 基于真实世界中 Claude 大模型的使用数据，分析 AI 对美国劳动力市场的实际影响；
- 发现 AI 尚未引发系统性失业，但高暴露职业中年轻求职者的招聘出现放缓迹象；
- 程序员等高学历、高收入白领岗位的任务被 AI 覆盖率较高（约 75%），而厨师、救生员、洗碗工等需物理交互的职业基本处于“零暴露”状态。

AI替代率



AI影响最大的职业 Top10

AI Exposure Ranking · Anthropic Research

1	程序员	74.5%
2	客服人员	70.1%
3	数据录入员	67.1%
4	医疗记录管理	66.7%
5	市场研究与营销	64.8%
6	销售代表	62.8%
7	金融与投资分析师	57.2%
8	软件测试（QA）	51.9%
9	信息安全分析师	48.6%
10	IT技术支持	46.8%

第1讲　软件与软件工程

1.6 开源软件



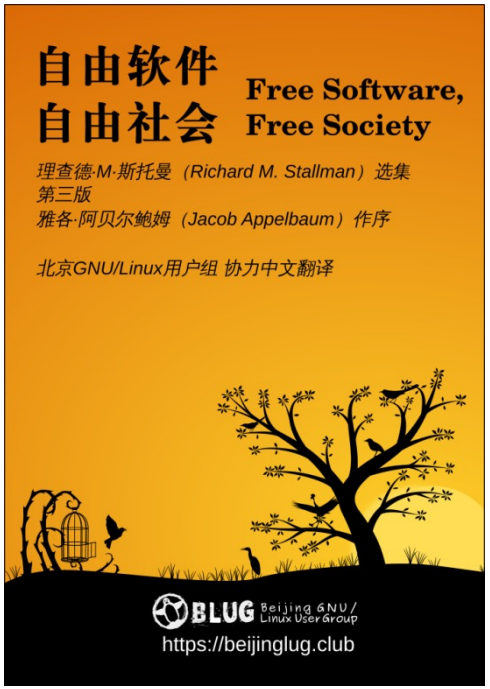
开源软件（Open Source）

- 开源是一种促进最终产品（通常是软件或程序）的免费访问和分发的理念。
- 开源软件是一种源代码可以任意获取的计算机软件，这种软件的版权持有人在软件协议的规定之下保留一部分权利并允许用户学习、修改以及以任何目的向任何人分发该软件。
- 开源社区又称开放源代码社区，一般由拥有共同兴趣爱好的人所组成，根据相应的开源软件许可证协议公布软件源代码的网络平台，同时也为网络成员提供一个自由学习交流的空间。
- 开源软件通常属于GNU许可，也有其它许可，如英特尔开源许可、FreeBSD许可、Mozilla公共许可。



自由软件（free software ）

- FSF (Free Software Foundation)的定义，自由软件表示的是那些赋予用户运行、复制、分发、学习、修改并改进软件这些自由的软件。
- 软件的原罪：
  - 软件私有一宗罪：浪费社会财富。
  - 软件私有二宗罪：不利于软件的使用。
  - 软件私有三宗罪：不利于软件的开发。
  - 软件私有四宗罪：不利于软件开发者学习。



免费软件（ freeware ）

- 免费软件就是免费提供给用户使用的软件，但是其免费的时候，通常都会有其他的限制，比如其源码不一定会公开，而且使用者也并没有使用、复制、研究、修改和再散布的权利。
- 开源软件是指在软件发行的时候，附上软件的源代码，并授权允许用户更改/自由再散布/衍生著作，开源并不抵制商业收费。

开源社区

Open Source Software Community or Open Source Community

- 开源中国（www.oschina.net/）

  OSCHINA ，Open Source China (OSC)，是国内最大的开源技术社区，拥有超过300万会员，形成了由开源软件库、代码分享、资讯、协作翻译、讨论区和博客等几大频道内容，为IT开发者提供了一个发现、使用、并交流开源技术的平台。
- codesoso 源码搜索（search.codesoso.com）

  在几十万的源代码中搜索，支持中文和英文搜索，可以出现很多代码，很适合刚学习编程或者是开始学习新的编程语言的小伙伴。
- CodeForge（www.codeforge.cn）

  源代码分享、下载网站，提供各种开发代码。
- github（github.com）

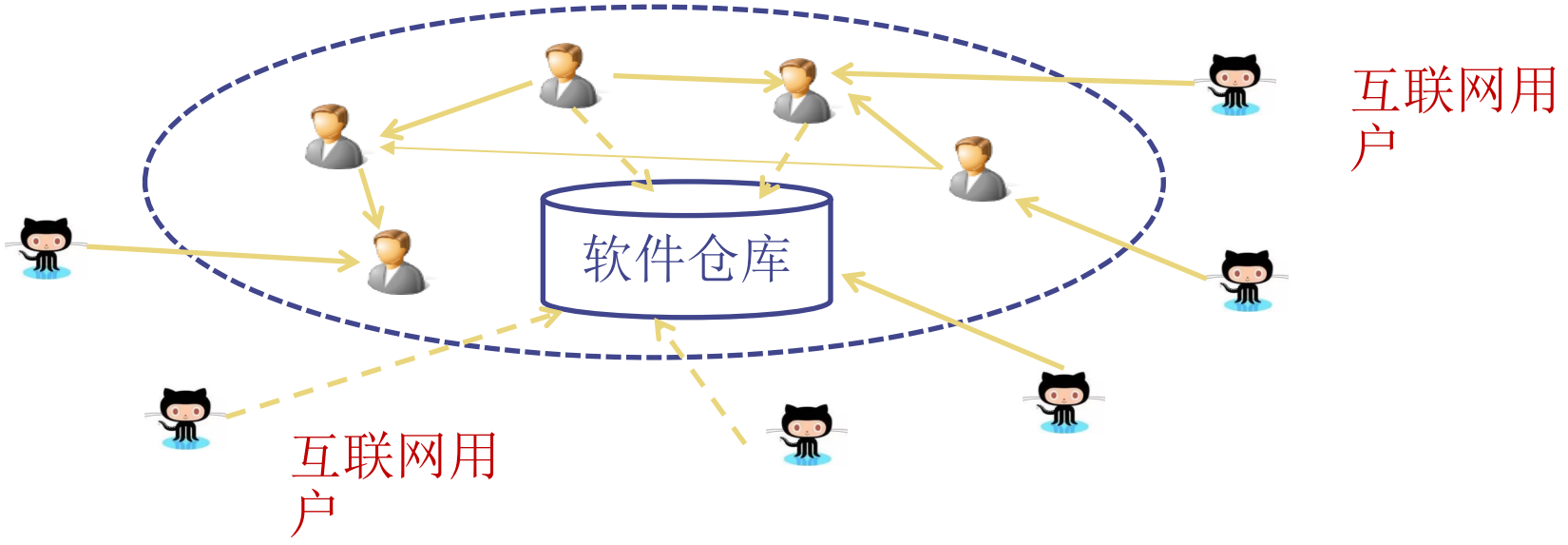
  全球知名开发者空间，聚集了超过两千万的程序员。交流、分享源代码。2018年6月4日，微软宣布，通过75亿美元的股票交易收购代码托管平台GitHub。

开源代码使用及风险

- 开源软件复用有助于缩减开发成本，提高开发效率。80%以上IT企业的主要产品复用了开源代码，仅有低于3%的企没有使用任何开源软件。
- 截至2019年1月，GitHub 的开源项目已达1800个， OpeHub索引的已经超573亿行，StackOverflow的软件不低于1300万条。
- 开源软件复用风险：
  - 开源许可证侵权。开源软件通常附带了相应的许可证限制，常见的许可证有GPL、LGPL、Apache 2. 0和BSD-2-Clause等110多种。
  - 已复用的代码难以及时更新，可能带来安全风险。

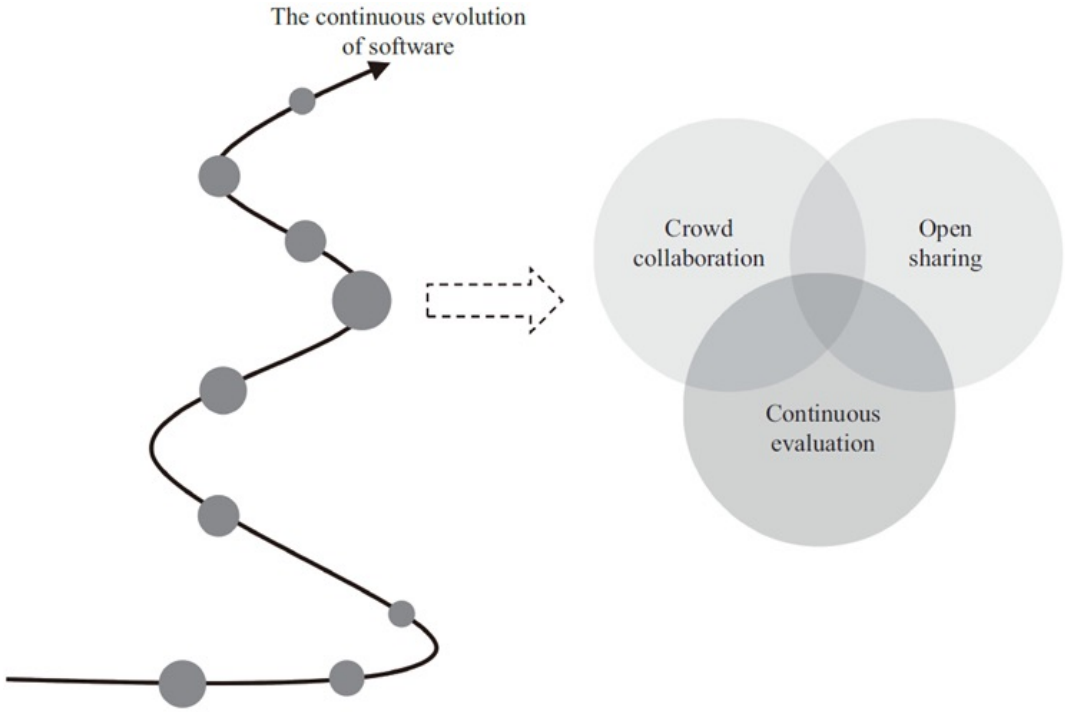
群智化开发方法

- 依托互联网平台来吸引、汇聚、组织和管理互联网上的大规模软件开发人员，通过竞争、合作、协商等多种自主协同方式，让他们参与软件开发、分享软件开发知识和成果、贡献智慧和力量的一种新颖软件开发方法.



软件众包

- **众包**（crowdsourcing）指的是一个公司或机构把过去由员工执行的工作任务，以自由自愿的形式外包给非特定的（而且通常是大型的）大众志愿者的做法。
- 众包是个人或者组织从大量相对**开放**且经常快速发展的参与者中获取包括想法、投票、微观任务等商品或服务的一种采购模式，众包通常涉及使用互**联网**或**激励**来吸引参与者并在参与者中发布任务。



第一讲 小结

1. 软件，软件的特点
2. 软件工程，软件工程的特点（与其它工程的区别）
3. 软件开发存在的问题
4. 软件工程的学科的知识构成
5. 软件工程的基本原理
6. 软件生存周期
7. 软件生存周期模型（瀑布、演进、迭代、增量、原型、螺旋、RUP、形式化）